

Lab Steps

Step 1: Review Previous Work and Floor Plan Context

The floor plan stage begins after the synthesis flow has been completed. From the previous session, you should have completed:

- Design setup
- Synthesis flow
- Flop count verification
- Clock percentage analysis

Step 2: Understand Floor Plan Objectives

Floor plan involves the following key activities:

- **Die and Core Area Definition:** Setting the overall chip boundaries and usable core area
- **Aspect Ratio Configuration:** Defining the height-to-width ratio of the core
- **Utilization Factor:** Specifying how densely cells should be packed
- **Input/Output Cell Placement:** Positioning I/O cells around the core periphery
- **Power Distribution Network (PDN):** Creating power and ground distribution structures
- **Macro Placement:** Positioning large pre-designed blocks

Standard cell placement occurs in a later stage and is not performed during floor planning.

Step 3: Access Configuration Files

Navigate to the OpenLane configuration directory:

```
cd openlane/configuration
```

```
pwd
```

```
ls -la
```

Open and review the README file to understand the available variables for each design stage.

```
cat README.md
```

Step 4: Understand Global Configuration Variables

The configuration hierarchy defines parameter precedence. Review the variables section in the README, which includes:

- **Global Variables:** Design name, Verilog files
- **Synthesis Variables:** Library files, driving cell, design rule constraints (transition max, span out, max capacitive load), minimum and maximum libraries for static timing analysis
- **Floor Plan Variables:** Core utilization, aspect ratio, core margin, I/O metal layers, PDN parameters
- **Placement Variables:** Target density and related settings

Step 5: Review Floor Plan Specific Switches

Key floor plan configuration parameters include:

FP_CORE_UTIL (Floor Plan Core Utilization)

This parameter defines the core utilization percentage—the ratio of total cell area to total core area. Default value is 50%.

```
FP_CORE_UTIL = 50
```

FP_ASPECT_RATIO (Aspect Ratio)

This parameter sets the height-to-width ratio of the core. Default value is 1 (square core).

```
FP_ASPECT_RATIO = 1
```

FP_CORE_MARGIN (Core Margin)

This parameter specifies the offset between the die boundary and the core boundary. Default value is 3.6 units.

FP_CORE_MARGIN = 3.6

FP_IO_HMETAL and FP_IO_VMETAL (I/O Metal Layers)

These parameters define the horizontal and vertical metal layers used for I/O placement. They are specified as follows:

FP_IO_HMETAL = 3

FP_IO_VMETAL = 4

Note: In the OpenLane flow, the actual metal layer used will be one layer higher than specified (e.g., if FP_IO_VMETAL is 4, metal layer 5 will be used; if FP_IO_HMETAL is 3, metal layer 4 will be used).

*FP_PDN_ (Power Distribution Network Parameters)**

These variables configure the PDN. The PDN_CORE_NET specifies the decoupling capacitors to be used during floor planning.

FP_PDN_CORE_NET = <capacitor_specification>

Step 6: Understand Placement Configuration Parameters

PL_TARGET_DENSITY (Placement Target Density)

This parameter controls how closely or how spread out cells should be packed during initial placement:

- Value of 1: Design is very tightly packed with minimal free space
- Value close to 0: Design is spread out with more free space

Default is set close to 0 for initial loose packing. This allows for congestion analysis before gradually tightening the design toward closure.

PL_TARGET_DENSITY = 0.x

Step 7: Examine GPIO Mode Configuration

FP_PIN_ORDER_CFG (Pin Order Configuration)

The GPIO_MODE parameter controls pin positioning around the core:

- Mode 1: Pin positions are random but at equal distances
- Mode 0: Pin positions are not equally spaced

This configuration affects how input/output cells are distributed around the core perimeter.

GPIO_MODE = 1

Step 8: Navigate to Design Directory

Change to your design-specific directory:

```
cd ../designs/picorv32a
```

```
ls -ltr
```

Step 9: Review Design-Specific Configuration Files

You should see two design-specific configuration files:

- config.tcl
- sky130.tcl

These files override system default settings. The configuration precedence is:

1. System defaults (lowest priority)
2. config.tcl (design-specific overrides)
3. sky130.tcl (technology-specific overrides) (highest priority)

Step 10: Examine config.tcl File

Open the design configuration file:

```
cat config.tcl
```

Review the following sections:

Design Name and Source Files

```
set ::env(DESIGN_NAME) "picorv32a"
```

```
set ::env(VERILOG_FILES) "./designs/picorv32a/src/picorv32a.v"
```

```
set ::env(SDC_FILES) "./designs/picorv32a/src/picorv32a.sdc"
```

Clock Configuration

```
set ::env(CLOCK_NET) "5.000"
```

```
set ::env(CLOCK_PORT) "clk"
```

Floor Plan Parameters

```
set ::env(FP_CORE_UTIL) 65
```

```
set ::env(FP_IO_VMETAL) 4
```

```
set ::env(FP_IO_HMETAL) 3
```

These settings override the system defaults. In this example:

- Core utilization is set to 65% (overriding the default 50%)
- I/O vertical metal is set to 4 (which will use metal layer 5 in the flow)
- I/O horizontal metal is set to 3 (which will use metal layer 4 in the flow)

Step 11: Compare System Defaults with Design Configuration

From the system default configuration (openlane/configuration/floorplan.tcl):

- FP_CORE_UTIL default: 50
- FP_IO_VMETAL default: 2 (resulting in metal layer 3)
- FP_IO_HMETAL default: 3 (resulting in metal layer 4)

From the design-specific configuration (config.tcl):

- FP_CORE_UTIL: 65
- FP_IO_VMETAL: 4 (resulting in metal layer 5)
- FP_IO_HMETAL: 3 (resulting in metal layer 4)

Step 12: Execute the Floor Plan Flow

Run the floor plan stage using the `run_flow_plan` command:

```
run_floorplan
```

Step 13: Monitor Execution and Review Output

During execution, you may see warnings such as obsolete statement warnings. These warnings can typically be disregarded as they refer to outdated syntax that is no longer required.

[WARNING] <message>: The source statement is obsolete and no longer required

Step 14: Verify Successful Completion

Upon successful completion, the floor plan flow will finish without errors. The absence of error messages indicates that the floor plan stage has been completed successfully.

[INFO]: PDN generation was successful.

Step 15: Understanding Configuration Flexibility

The floor plan stage, like the overall PNR (Place and Route) flow, is iterative:

- **Initial Stages:** Parameters are set to prioritize congestion analysis with looser packing (lower target density)
- **Later Stages:** Parameters are adjusted to focus on timing closure with tighter packing
- **Designer Control:** You decide which parameters to activate at each iteration based on design requirements

This flexibility allows you to optimize the design progressively as you move toward final closure.

Key Takeaways:

- Floor plan configuration uses a three-level precedence hierarchy: system defaults, design-specific config, and technology-specific overrides
- Core utilization determines cell packing density
- I/O metal layers are specified one layer below their actual usage in the flow
- Placement target density controls initial cell distribution for congestion management
- The floor plan stage must be successfully completed before proceeding to placement and routing stages

Review floorplan files and steps to view floorplan

This lab demonstrates the verification and visualization of floorplan configuration in an open-source hardware design flow. The objective is to confirm that design-specific configuration parameters override system defaults, and then visualize the resulting floorplan layout. The lab involves navigating the runs folder, examining log files and configuration files to verify parameter precedence, inspecting the DEF (Design Exchange Format) file to understand chip dimensions, and finally opening the floorplan in Magic for visual inspection.

Lab Steps

Step 1: Navigate to the Runs Folder

Navigate to the runs folder to access the latest generated design files.

```
cd runs
```

Check the contents to locate the folder corresponding to the current date (e.g., 10th September).

```
ls -ltr
```

Step 2: Verify Configuration Parameter Precedence

Move into the logs directory for the current run to examine the floorplan logs.

```
cd logs
```

```
cd floorplan
```

Step 3: Examine IO Placer Log File

Open the IO placer log file to verify that design-level configuration parameters have overridden system defaults. Specifically, check the metal layer assignments.

Open the io_placer.log file (or similar)

```
cat ioPlacer.log
```

Search for metal layer information using shift+G to navigate within the file viewer.

The log should display:

- **Vertical metal layer:** 5 (set by design config)
- **Horizontal metal layer:** 3 (set by design config)

Step 4: Compare with Design Configuration File

Navigate to the design folder and examine the config.tcl file to verify the source configuration values.

```
cd ../../design
```

```
cat config.tcl
```

Verify the configuration contains:

- **Vertical metal:** 4 (which becomes $4 + 1 = 5$ in the floorplan)
- **Horizontal metal:** 3 (which becomes $3 + 1 = 4$ in the floorplan)
- **Core utilization:** 65%

Step 5: Check Configuration Priority in Run-Specific Config

Navigate to the run configuration file to understand the parameter precedence hierarchy.

```
cd ../runs/[DATE]/config.tcl
```

```
cat config.tcl
```

Note the priority hierarchy (from highest to lowest priority):

1. Run-specific config.tcl
2. Design config.tcl
3. System defaults

For example, core utilization should show the value of 50 at the run level, which overrides the 65% from the design config.tcl.

Step 6: Navigate to Floorplan Results

Move to the results directory to access the DEF file containing the floorplan data.

```
cd ../results
```

```
cd floorplan
```

```
ls -la
```

tep 7: Examine the DEF File

Open and inspect the DEF (Design Exchange Format) file to understand the chip dimensions and cell placements.

```
cat *.def
```

![PLACEHOLDER: screenshot showing DEF file header and die area specification]

Locate the die area definition in the DEF file. The syntax is:

```
DIEAREA (X1 Y1) (X2 Y2) ;
```

Where:

- **(X1, Y1)** = lower left corner coordinates (typically 0, 0)
- **(X2, Y2)** = upper right corner coordinates

For example:

```
DIEAREA (0 0) (500000 400000) ;
```

Step 8: Calculate Die Dimensions

Use the database unit specification to convert DEF coordinates to actual chip dimensions.

The DEF file contains a units statement:

```
UNITS DISTANCE MICRONS 1000 ;
```

This indicates that **1 micron = 1000 database units**.

To calculate actual dimensions in micrometers, divide the coordinates by 1000:

Chip width = $(X2 - X1) \div 1000$ micrometers

Chip height = $(Y2 - Y1) \div 1000$ micrometers

Step 9: Open Floorplan in Magic Layout Editor

Launch Magic to visualize the floorplan layout. First, identify the tech file location and construct the Magic command.

The tech file is typically located at a path similar to:

.../tech/sky130_open_pdk_public_release_tech_file.tech

Navigate to the results directory containing the DEF file and run Magic:

```
magic -T  
/home/vsduser/Desktop/work/tools/openlane_working_dir/pdks/sky130A/libs.tech/sky  
130A.tech lef read ../../tmp/merged.lef def read picorv32a.floorplan.def &
```


Step 10: Visualize the Floorplan

Once the files are loaded in Magic, the floorplan visualization will display:

- All placed cells and their orientations
- Metal layers and routing tracks
- Die boundaries
- IO placement around the perimeter

Use Magic's navigation tools (zoom, pan) to inspect different regions of the floorplan and verify that the configuration parameters have been correctly applied during floorplanning.

Review floorplan layout in Magic

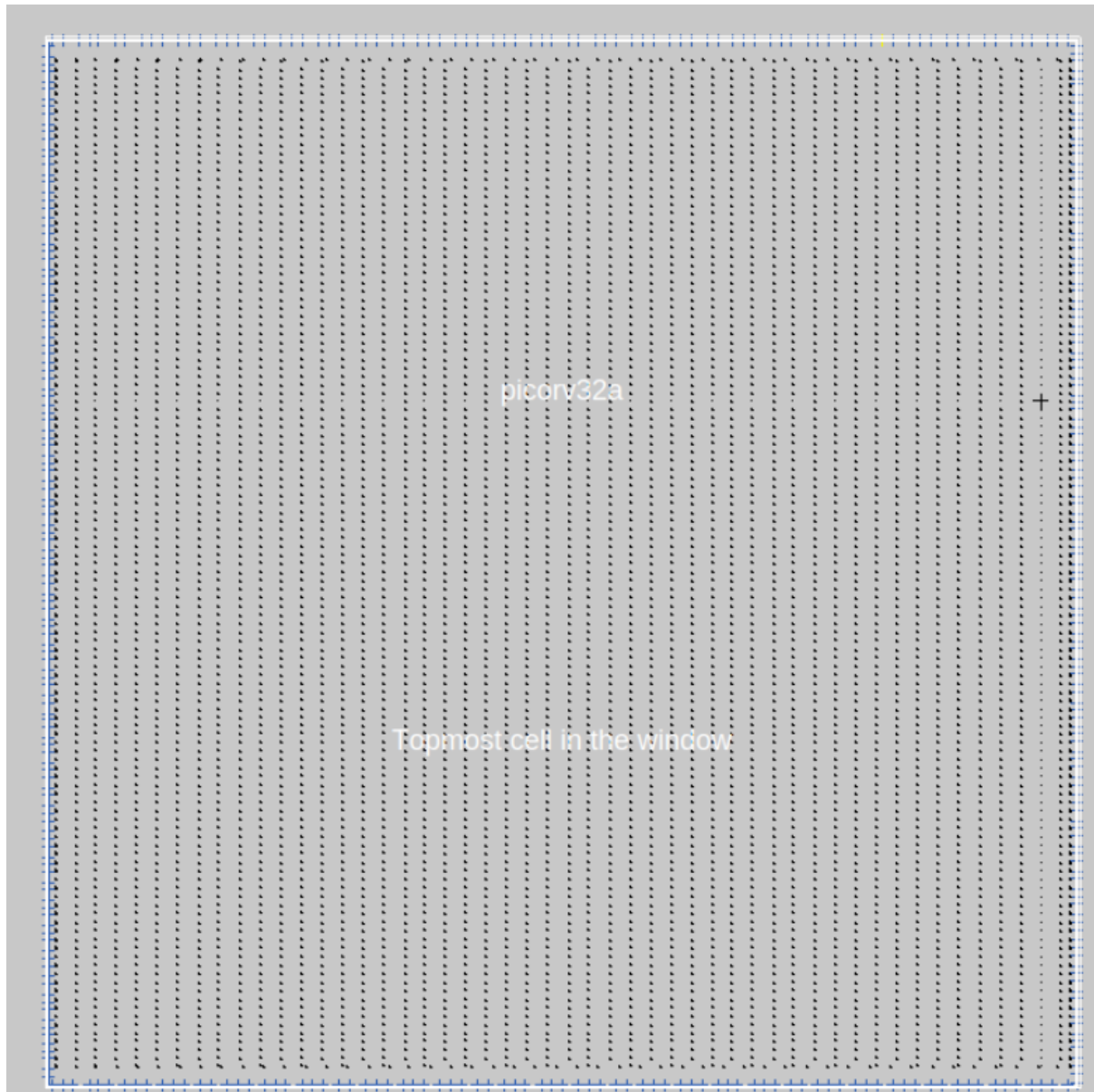
This lab focuses on navigating and analyzing the layout window of an integrated circuit design using keyboard shortcuts, mouse actions, and on-screen inspection tools. The aim is to:

- Center and fit the layout on screen.
- Zoom into layout portions and inspect object properties.
- Identify pin placement and layers.
- Examine device boundary structures and special cells (TAP, standard cells).
- Explore the effect of configuration changes on the layout.

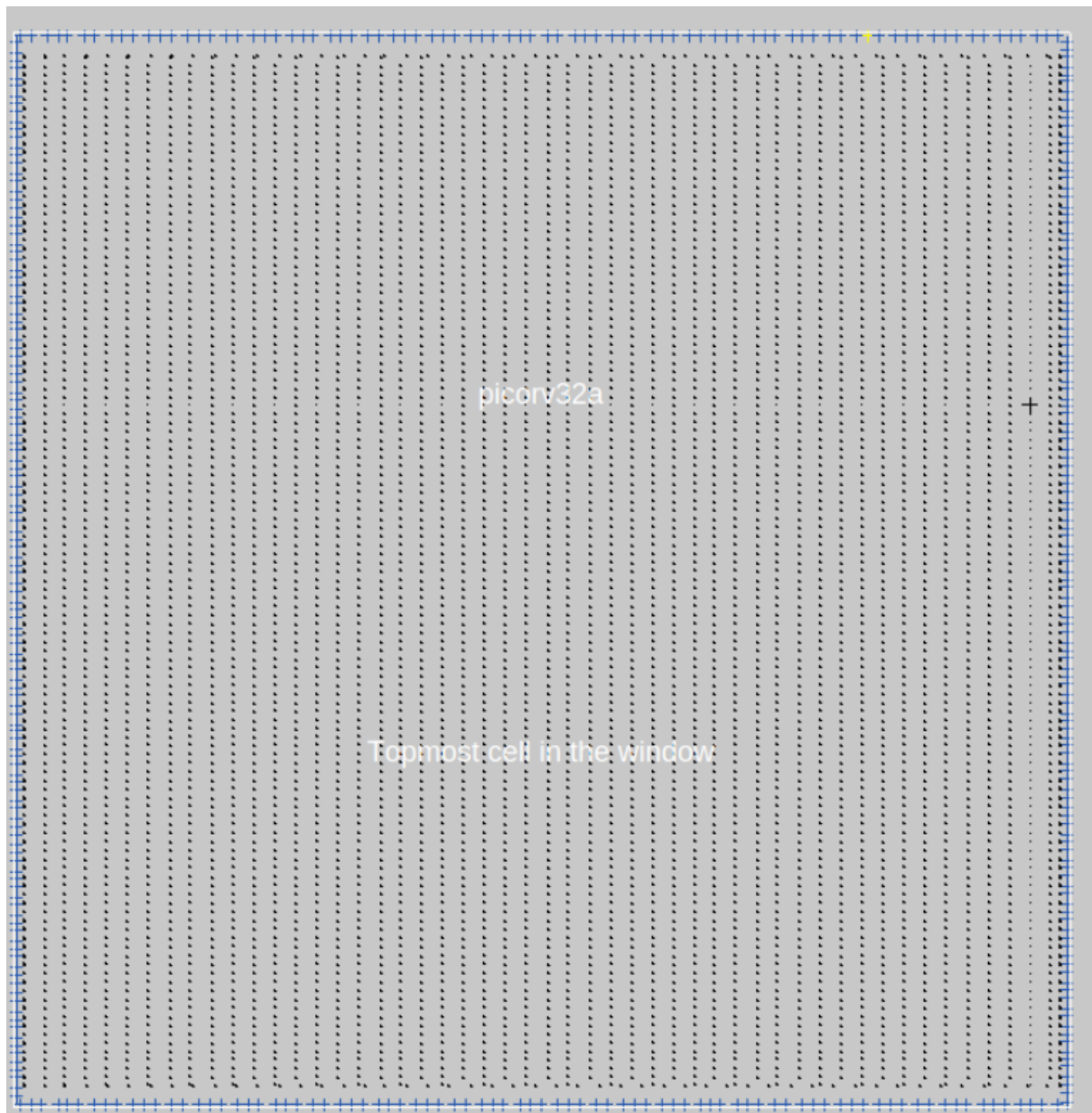
Lab Steps

1. Center and Fit the Layout Window

- **Select** the entire layout:
 - Press S on your keyboard.
 - The whole layout will be **highlighted**.

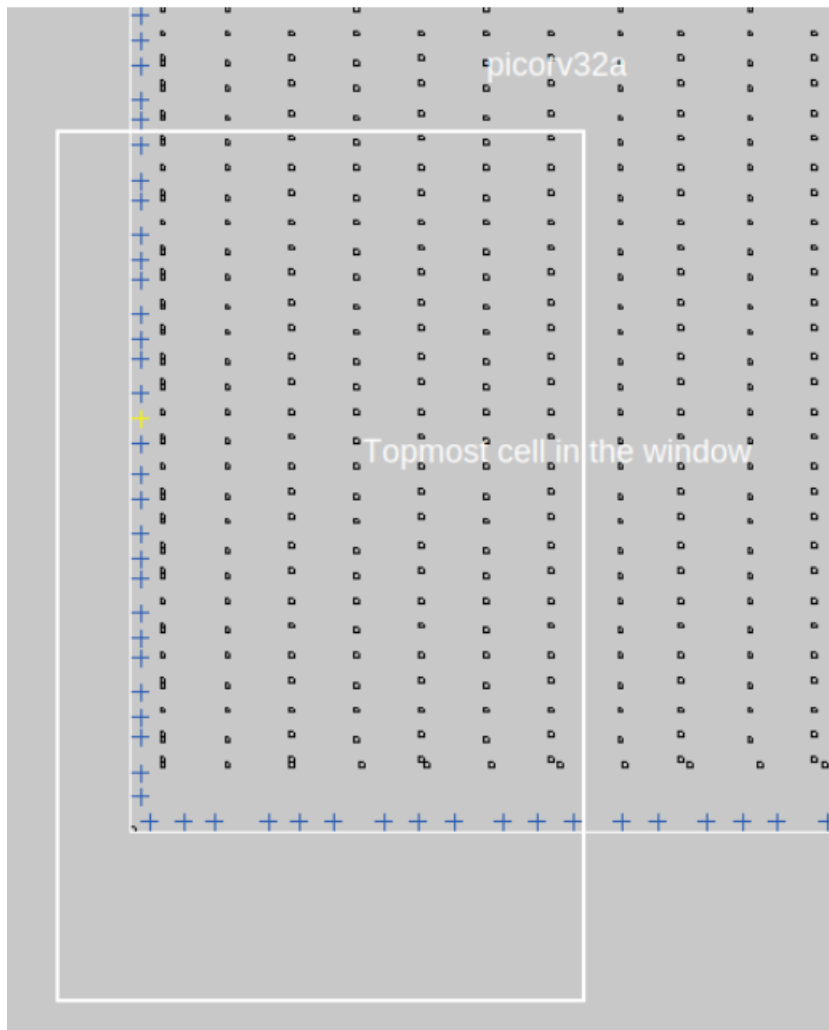


- **Fit** the selected layout to your display:
 - Press V on your keyboard.
 - The layout will now fill the screen area.

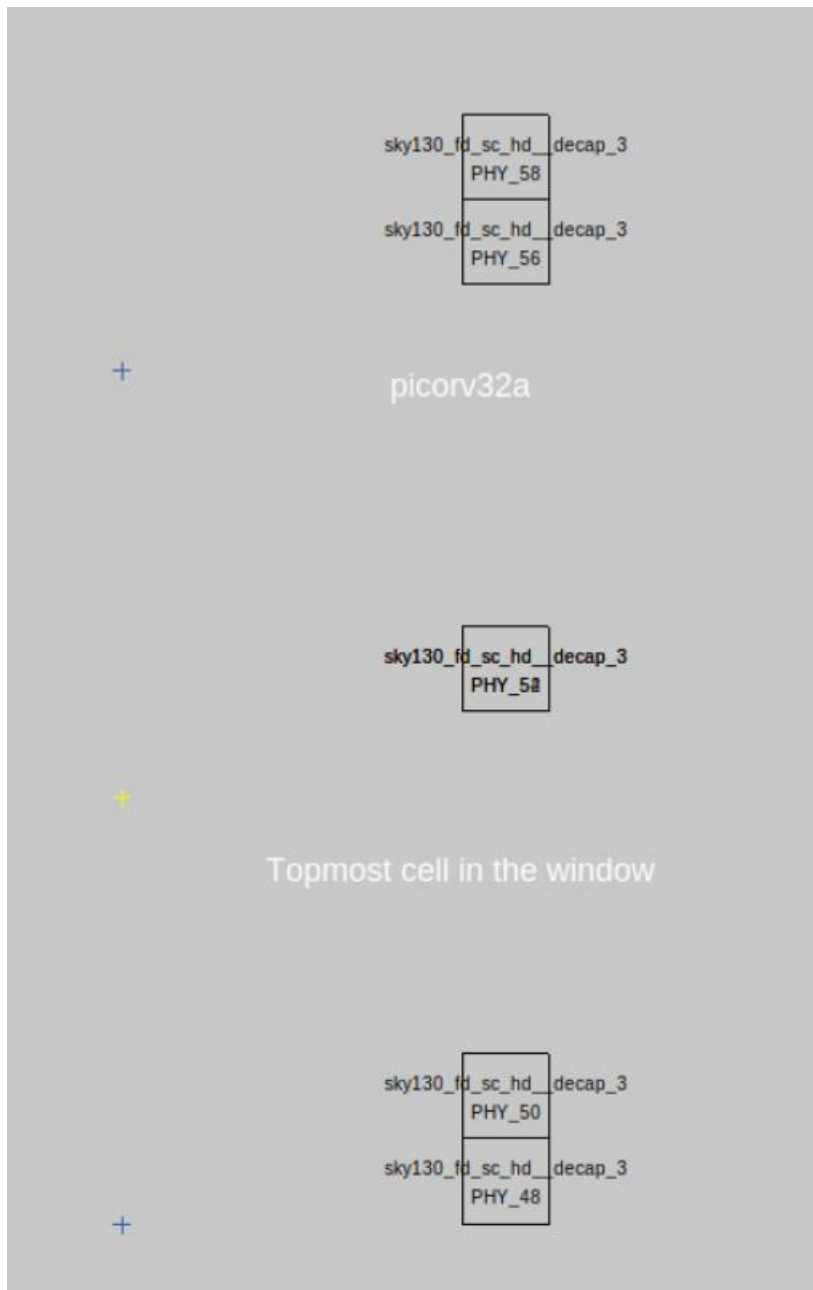


2. Zooming and Navigating in the Layout

- **Zoom into a Region:**
 - Do a **left mouse click** at the starting corner of your zoom box.
 - Do a **right mouse click** to define the opposite corner.
 - Press Z on your keyboard.
 - The selected area will zoom in.

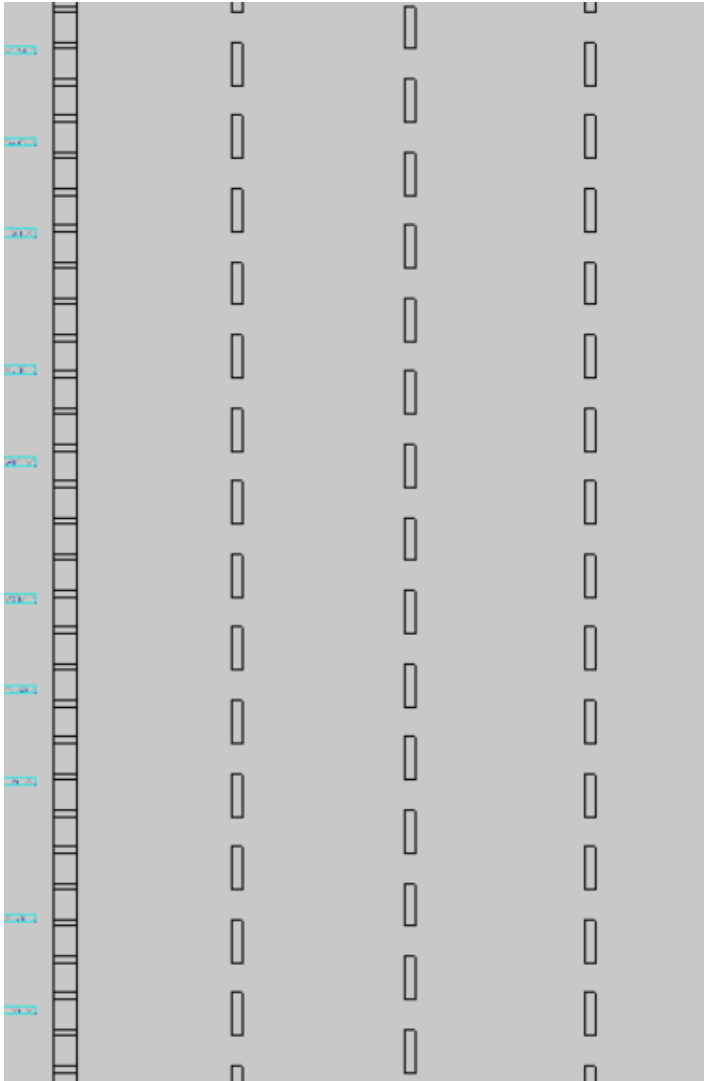


Repeat zoom by pressing Z again to incrementally zoom further.

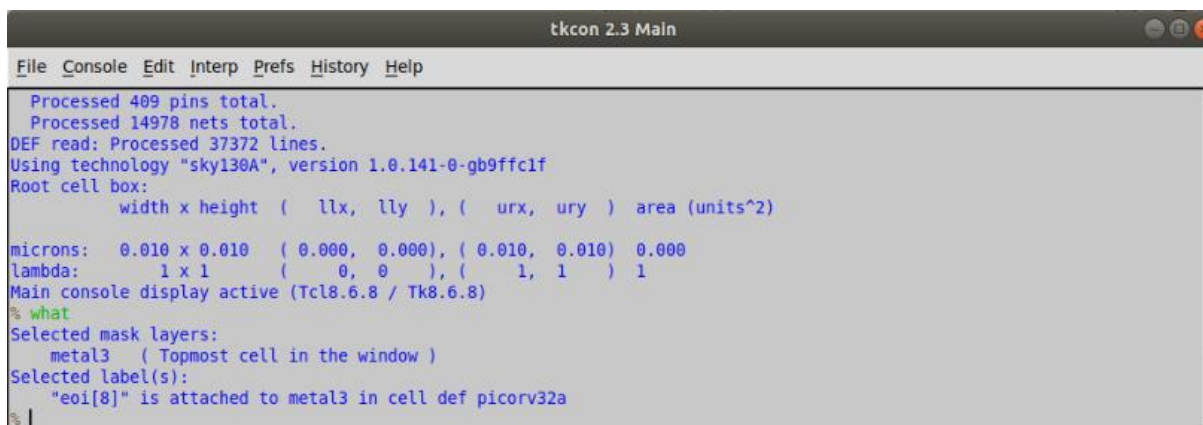
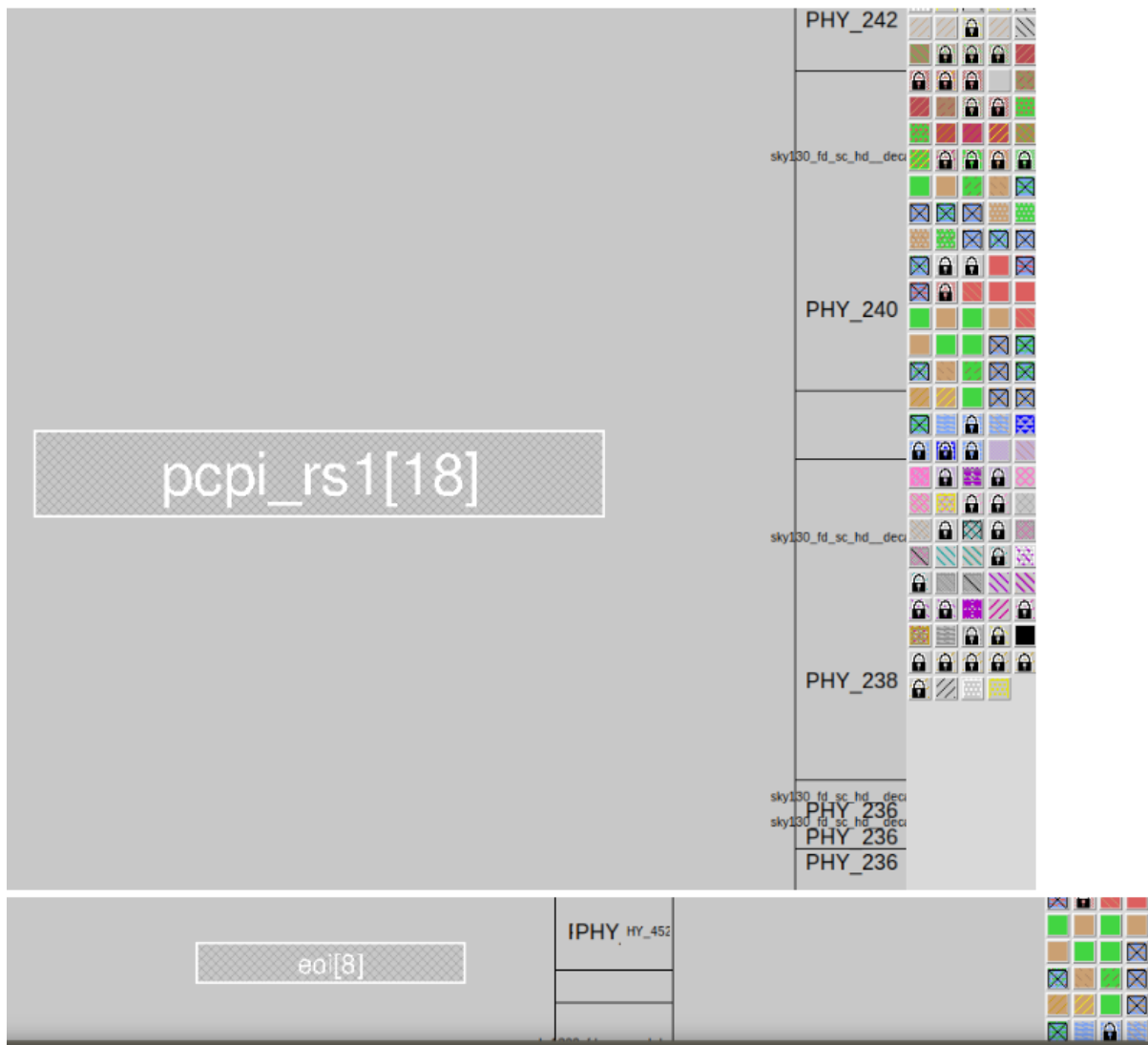


Inspecting Pin Placement and Layer Information

- **Pin Mode Information:**
 - Confirm that pin mode is set (pin mode value '1' places IO pins equidistant).
 - Observe input/output pins arranged equidistantly around the boundary.

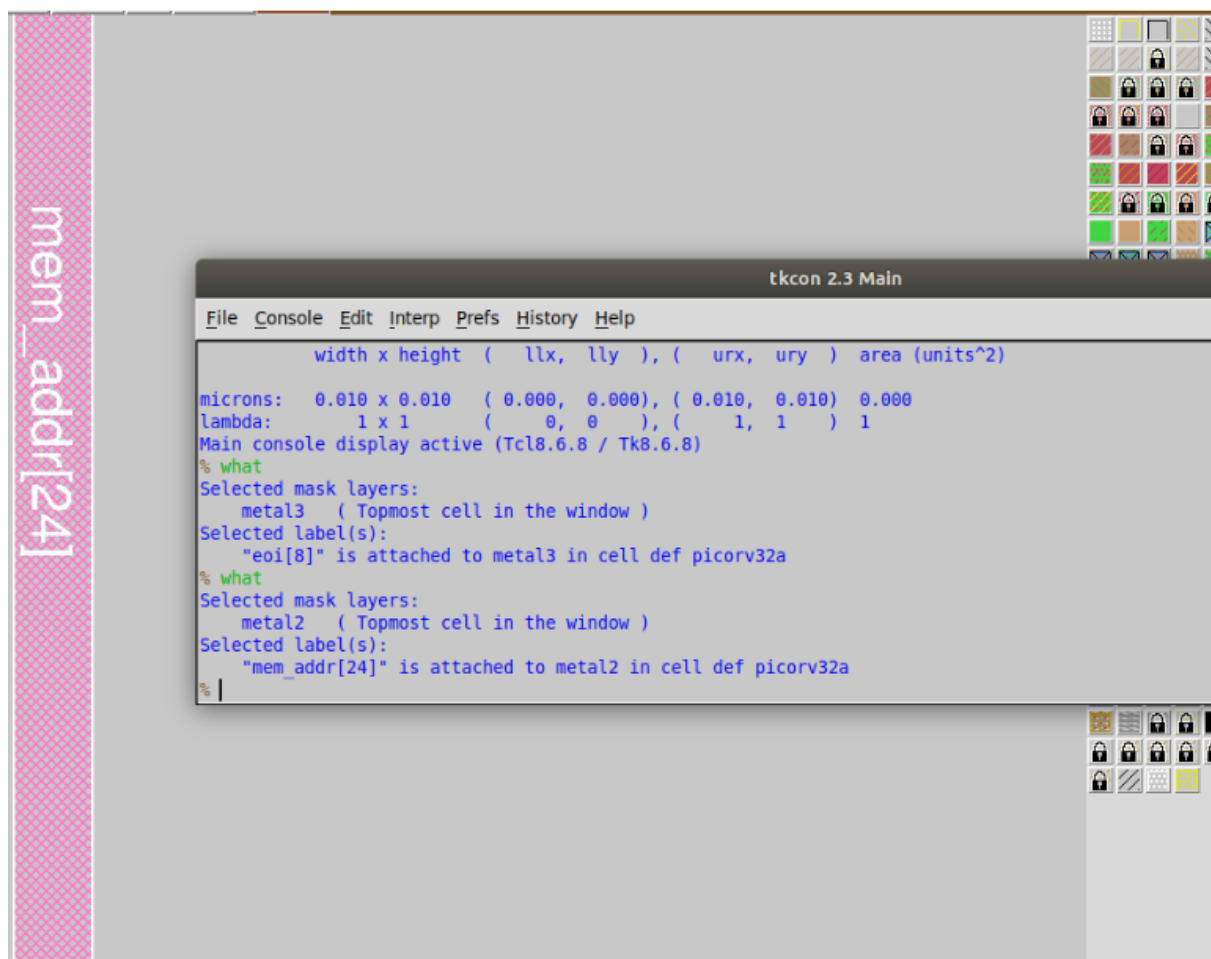
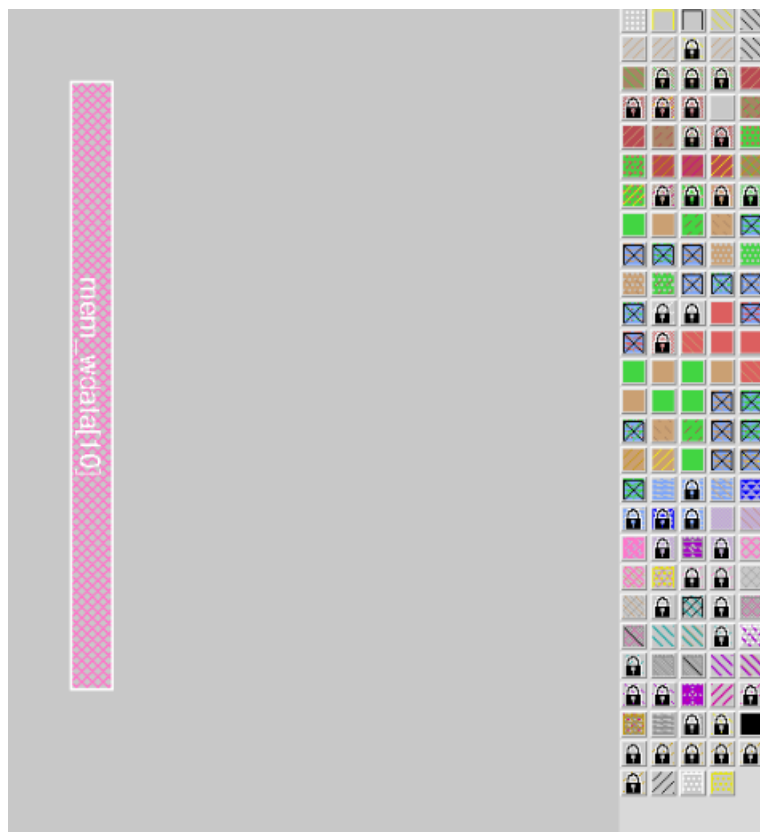


- **Check Pin Layer:**
 - Move your mouse cursor over an input or output pin (no click).
 - Press S to select the object.
 - A secondary **properties or Tekkon window** will appear, displaying the layer (e.g., "metal 3" for horizontal pins).



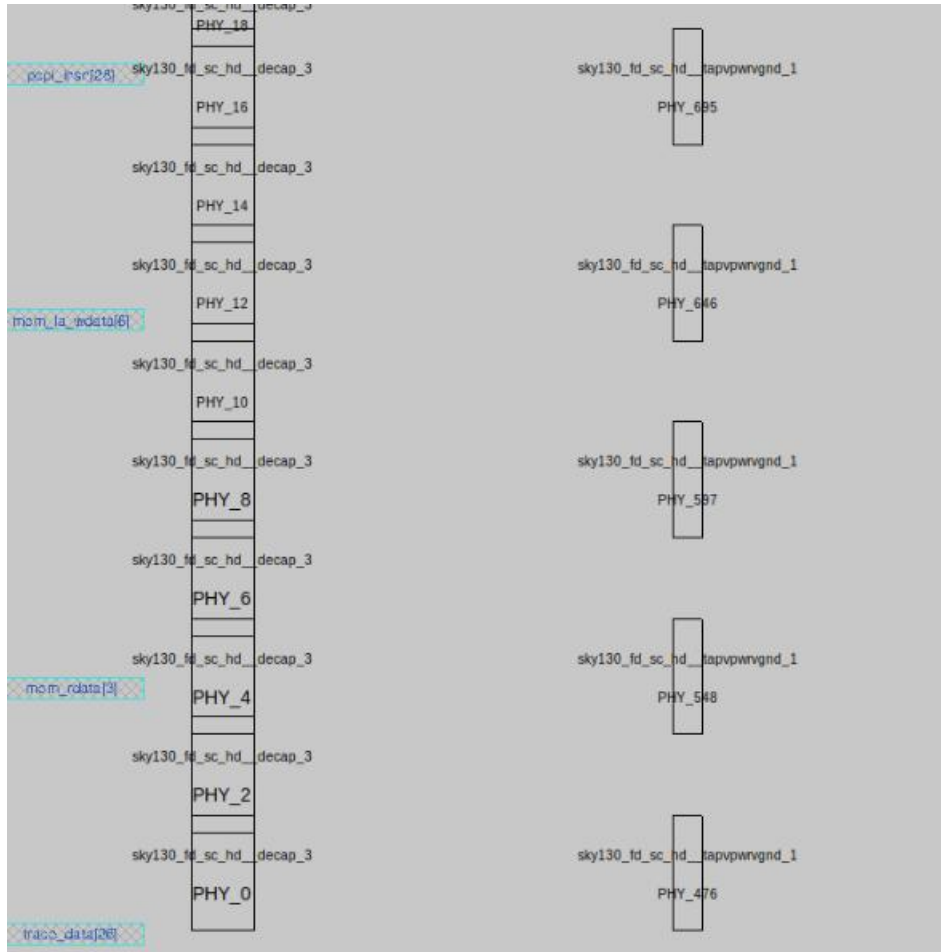
Check Vertical Pin Layer:

- Move over a vertical pin and press S.
- The window should show the layer as "metal 4".



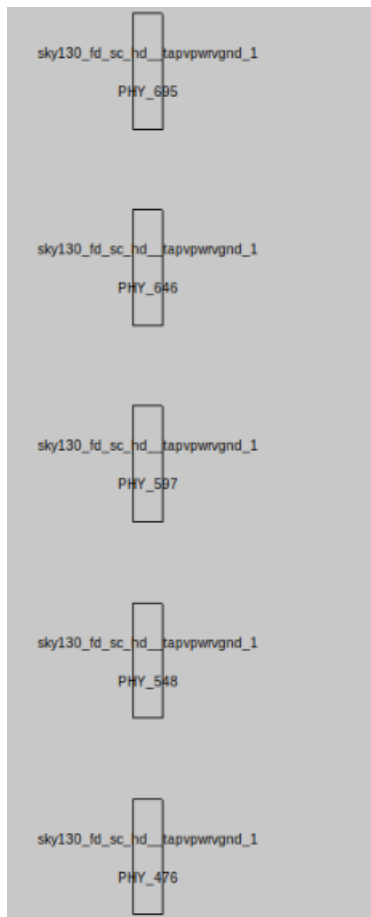
4. Identify Device Boundaries and Special Cells

- **D Capsules (Boundary Endcaps):**
 - View the layout edge to see D capsules aligned along the side rows, marking the device boundary.



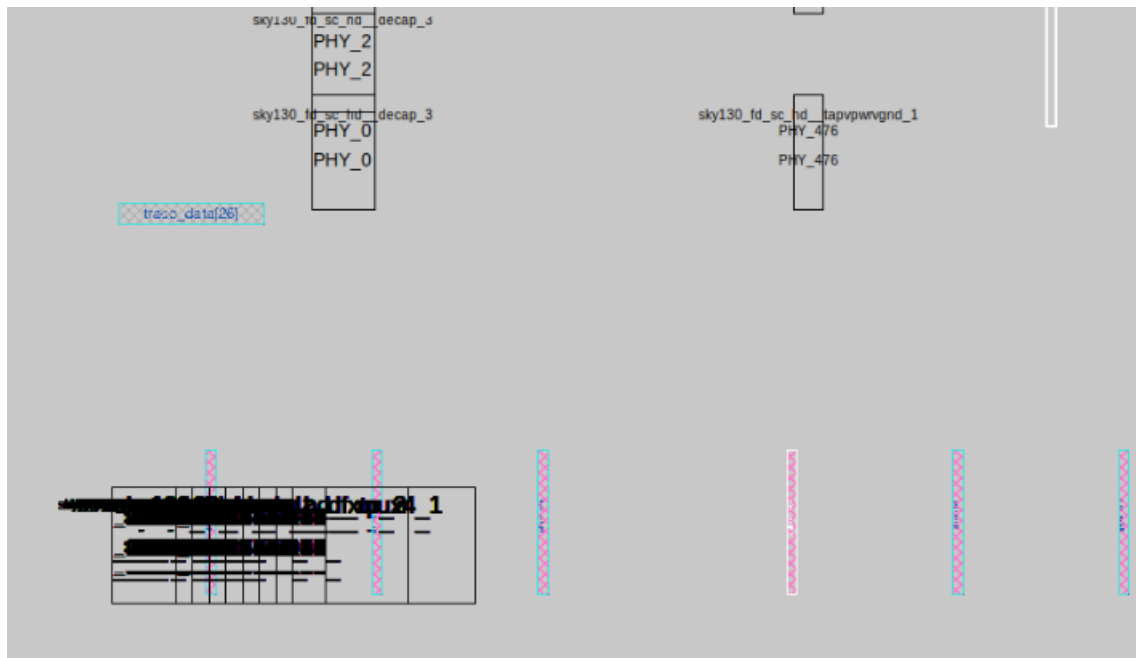
TAP Cells:

- Zoom in closer on the layout to spot TAP cells.
- TAP cells **connect the N-well to VDD and substrate to ground** to prevent latch-up in CMOS devices.
- These cells appear in a **diagonally equidistant arrangement**. Check the lab README/config file for the relevant switch controlling the TAP cell distance.



Recognize Floorplan Limitations and Identify Standard Cells

- **Floorplan Note:**
 - The floor plan stage **does not handle standard cell placement**. However, some standard cells may appear in corners.
- **Locate Standard Cells in Lower Corners:**
 - Look at the **lower left** (or right) region marked by dashed lines.
 - Use left click, right click, then zoom, and arrow keys to navigate closer.
 - Identify boxes corresponding to standard cells (e.g., "sky130 buffer", "clock buffer", "OR gate").
 - These standard cells are clustered in one region.

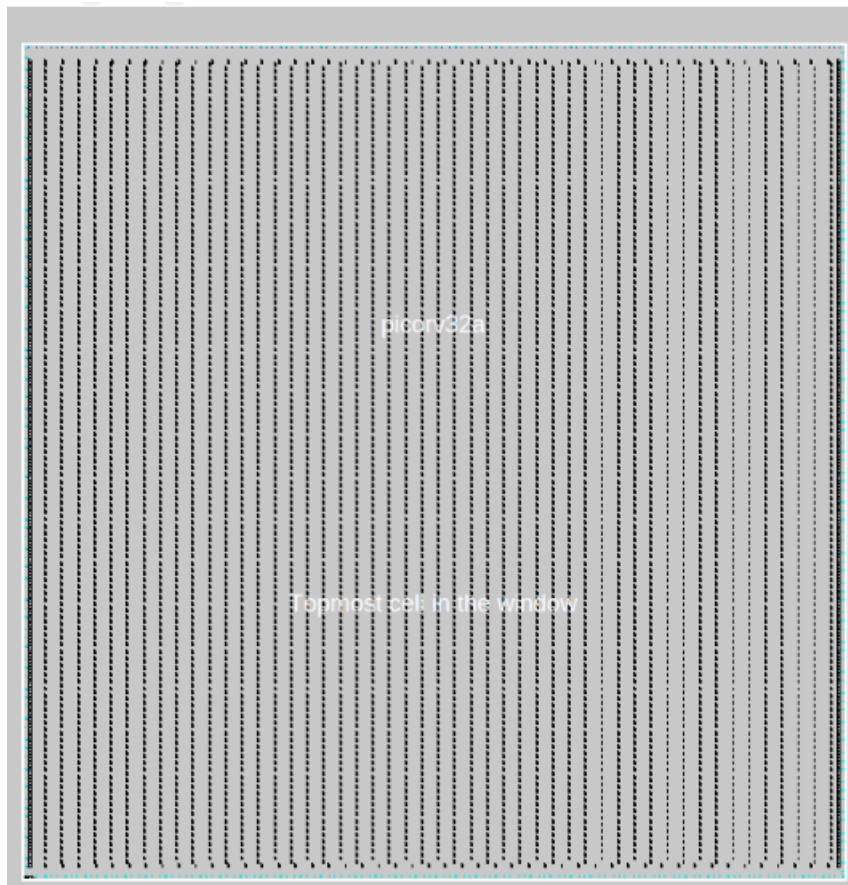


Experiment with Layout Configuration

- **Change IO Pin Setting:**
 - Adjust 'input/output pin' setting from 1 to 0 to alter placement.
 - Always make changes in the design-related configuration file for flexibility and traceability.

7. Refocus or Center Layout After Changes

- **Re-center and Fit Again:**
 - Press S, then V to highlight all and fit the layout again at any stage.



After completing these steps, you may close the layout window as needed and explore further with other display and configuration options for deeper familiarity.

Library Binding and Placement

Netlist binding and initial place design

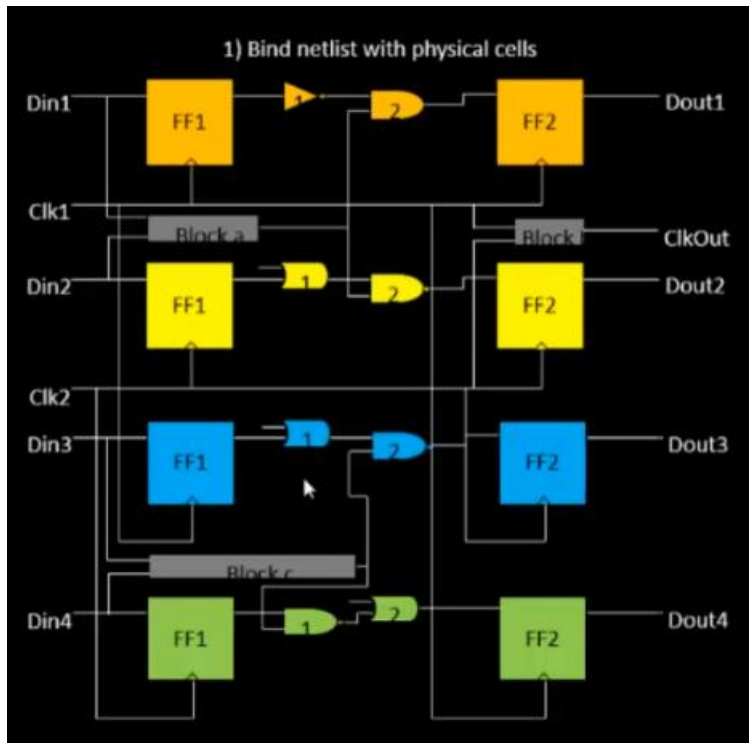
The placement and routing stage is a critical phase within the physical design flow for digital integrated circuits. This phase translates the logical representation of a netlist with abstract gates into an organized physical layout, assigning real, manufacturable shapes and positions to cells and interconnections based on defined libraries and constraints. It directly impacts the chip's performance, manufacturability, area, timing, and power consumption.

Core Concepts

Netlist to Physical Cells Mapping

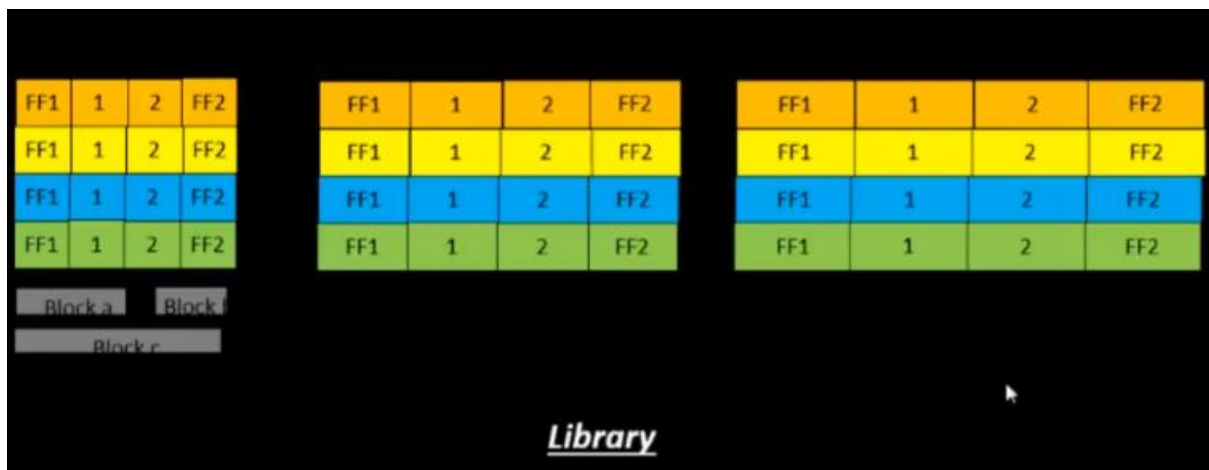
- Each gate or logic component in a netlist is mapped to a physical cell representation.
- Though symbolic shapes (like the traditional symbols for AND, OR, etc.) convey logical function, their physical counterparts are *rectangular or square blocks* with specific width and height for manufacturing.

- This mapping provides every netlist element with tangible, physical dimensions for placement.



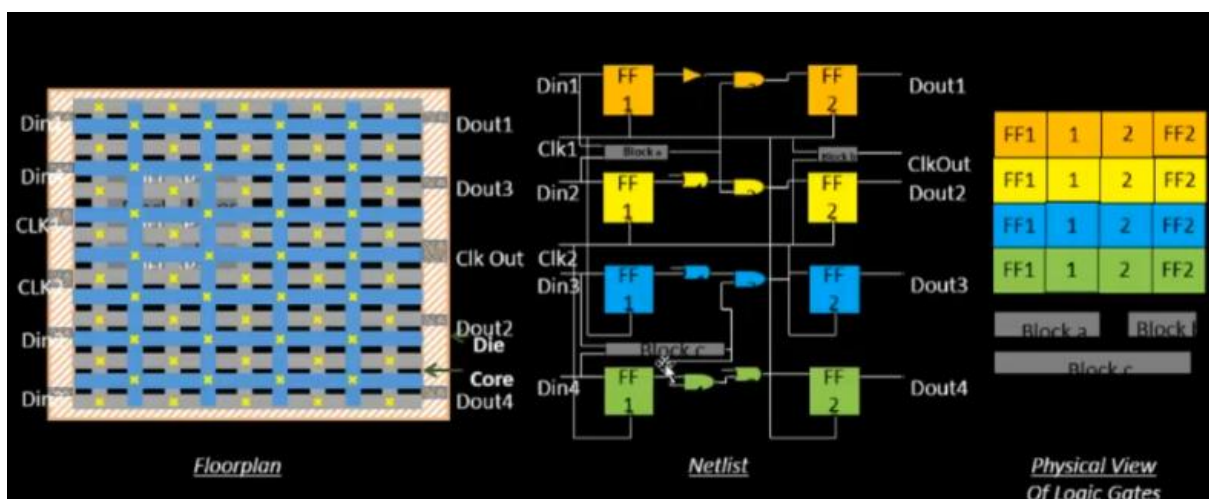
The Library

- A **library** is a collection that provides:
 - The *physical dimensions* (width and height) of every cell.
 - The *timing characteristics* (e.g., propagation delay).
 - The *behavioral conditions* (such as when a flip-flop produces output).
 - Multiple *flavors* (variants) of the same logic cell, offering trade-offs between area and delay (e.g., larger, faster cells vs. smaller, slower ones).
- Designers can select different flavors based on timing requirements and available silicon area.



Floorplanning and Cell Placement

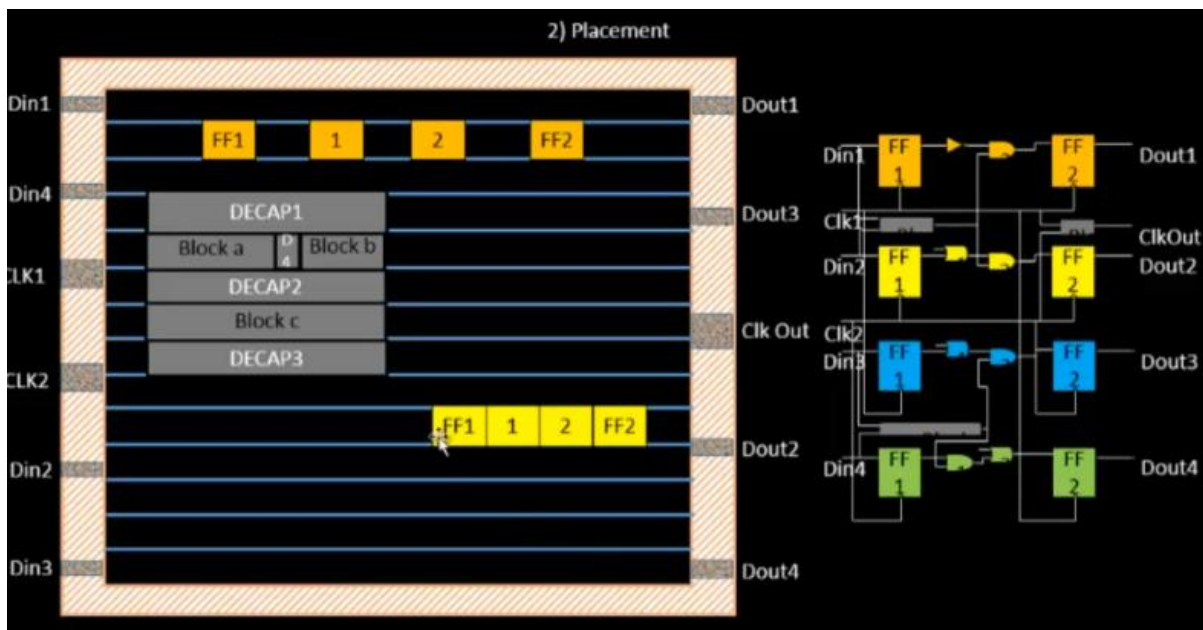
- The *floorplan* defines overall chip dimensions, as well as the locations of input/output ports and major macros.
- Placement involves arranging physical cells onto the defined floorplan *without overlap*, while considering:
 - Logical connectivity (keeping connected cells close to minimize interconnect delay).
 - Performance objectives (e.g., achieving low delay, minimal area, optimal power usage).
 - Design constraints such as legal regions and reserved areas that must not be used for cell placement.



Placement Objectives and Constraints

- Key goals during placement:

- Maintain proximity between logically connected components to minimize signal delay and wire length.
 - Prevent cells from occupying restricted or reserved regions (obeying block boundaries).
 - Satisfy specific timing and power requirements, possibly trading off area for higher performance cells where necessary.
- The physical arrangement directly affects communication delays between elements, critical for ensuring functional and timing correctness.



Real-world Optimization

- Multiple iterations and strategies are frequently required to balance design goals:
 - Compactness (minimizing area).
 - Minimizing power and signal crosstalk.
 - Ensuring manufacturability.
- The placement phase serves as the foundation for the subsequent routing step, which establishes metal interconnects following optimal paths defined by the component arrangement.

Key Points

- **Netlist binding** associates abstract logic elements with physical cells possessing real dimensions.

- A **cell library** provides options in terms of cell size, shape, and timing, supporting trade-offs for performance, area, and power.
- The **placement process** determines precise positions for each physical cell on the chip, ensuring no overlaps and optimal connectivity, while honoring reserved and special regions.
- Logical proximity in placement reduces communication delay; thus, connected modules (e.g., flip-flops to related I/O ports) should be placed near each other.
- **Variants of a cell** (different flavors) exist in the library to enable designers to choose between higher speed, larger area, or lower speed, smaller area as per design requirements.
- The placement phase prepares the groundwork for routing, where physical wiring interconnects all placed cells, respecting the placement, connectivity, and design rules.
- All these steps together directly influence the chip's speed, area, timing closure, and overall manufacturability.

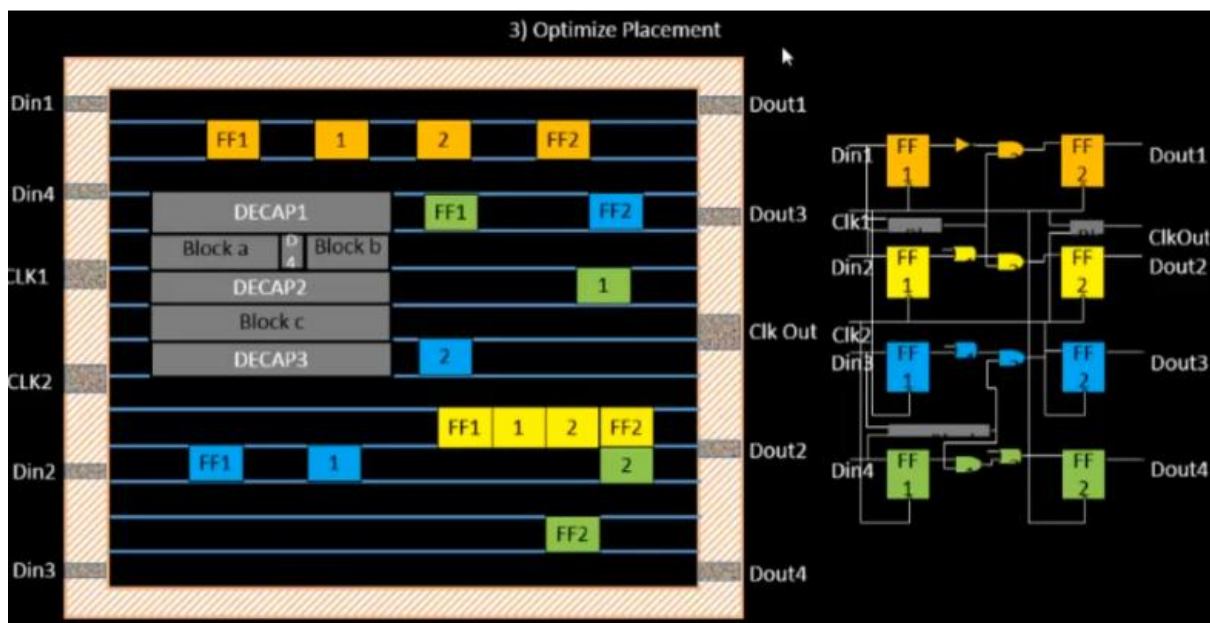
Optimize placement using estimated wire-length and capacitance

This section addresses theoretical principles of **cell placement** in hardware design, focusing on the geometric arrangement of flip-flops and logic gates relative to I/O ports and the consequences on signal integrity and placement optimization. It covers the impact of wire length on capacitance and resistance, the resulting signal degradation, and the necessity for **repeaters/buffers** to maintain signal integrity over long interconnects.

Core Concepts

Cell Placement and Floorplanning

Cell placement is the process of assigning physical locations for hardware elements (such as flip-flops and logic gates) within a design layout. The primary objective is to place related cells (like flip-flops and their associated logic) close to relevant I/O ports to minimize wire lengths, which directly impacts electrical characteristics.



- Placing **input flip-flops** closer to input ports improves signal receive quality.
- Placing **output flip-flops** closer to output ports enhances signal transmission.
- In situations with complex interconnections (e.g., logic separated by a large distance between I/O ports), this process becomes more challenging.

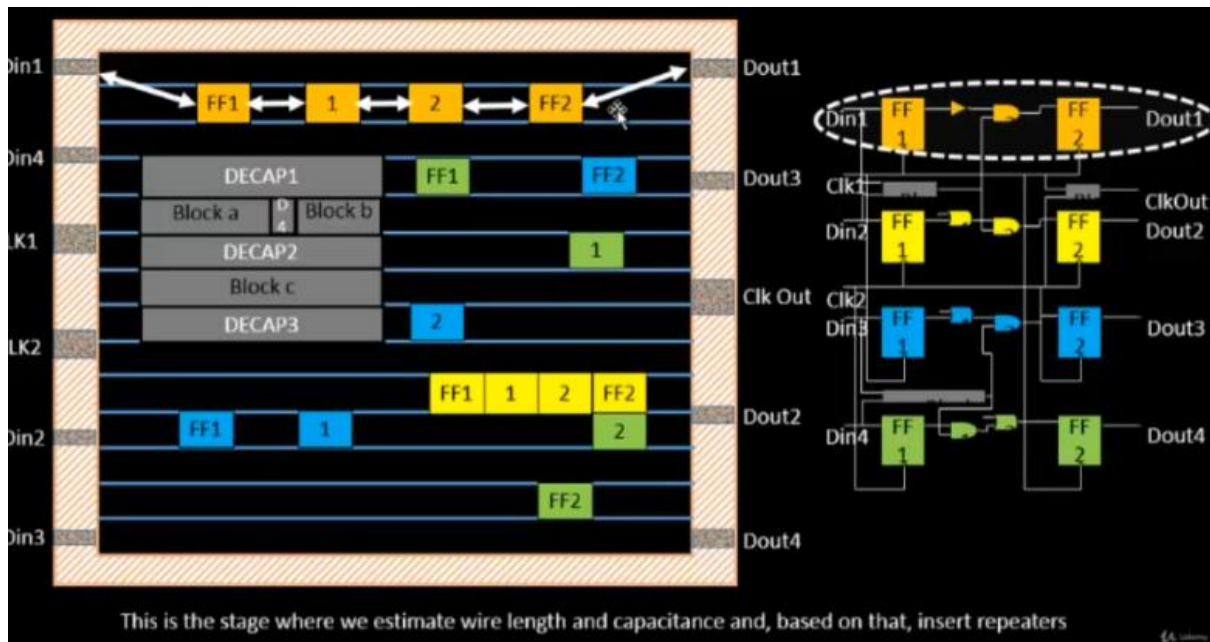
Impact of Wire Length

Long wires between cells introduce parasitic **capacitance** and **resistance**:

- **Capacitance (C):** For metal wires running over a substrate, the capacitance can be approximated by $C = \epsilon \frac{A}{d}$, where A is area, d is distance between plates (here, wire and substrate), and ϵ is the permittivity of the material.
- **Resistance (R):** Resistance of a wire is given by $R = \rho \frac{L}{A}$, with ρ as resistivity, L as length, and A as cross-sectional area.

Longer wires mean:

- Larger L causes increased R
- Increase in overall C
- Both degrade signal quality due to larger RC time constants.



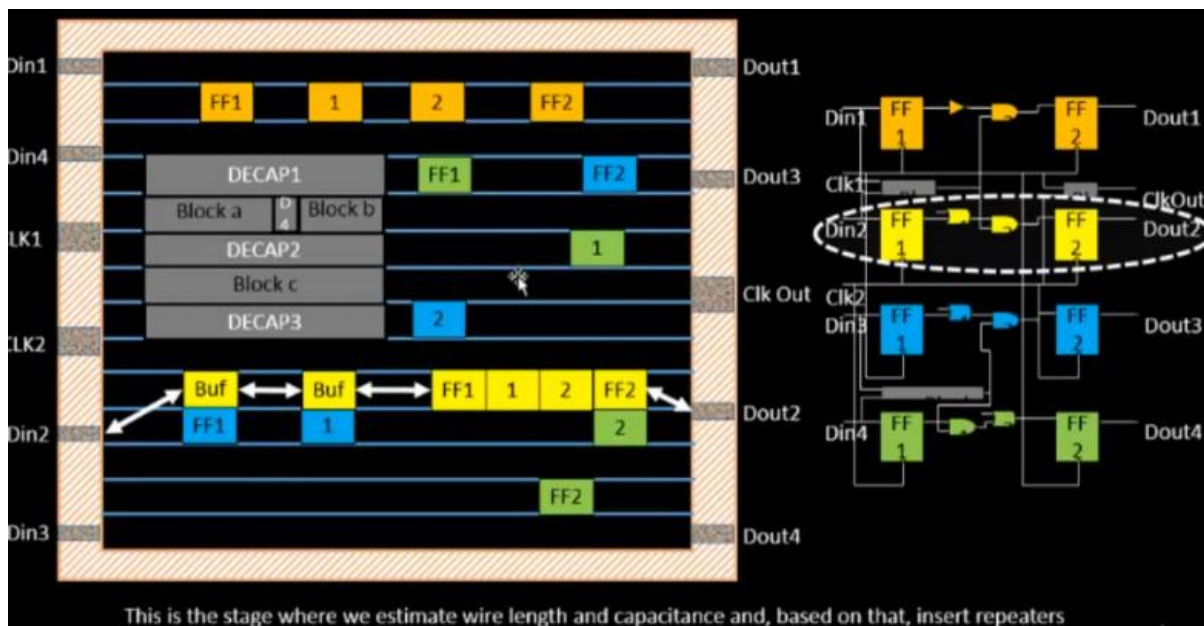
Signal Integrity and Repeaters

Signal integrity refers to the faithful transmission of digital signals without significant distortion or delay. As distance increases:

- High RC delays slow transitions (increase **slew**) and can cause signals to degrade before being captured at the receiver.
- If placement leads to excessive wire length, signal integrity is compromised.

Repeaters (Buffers):

- Repeaters are inserted along long wires to "refresh" the signal, re-driving it to restore full logic levels and fast transitions.
- Conceptually, breaking one long wire into several shorter segments, each driven by a buffer, reduces the RC timing penalty per segment.



Slew Rate and Capacitance

Slew rate measures the speed at which a signal transition occurs. It is highly dependent on capacitance:

- High C requires more charge ($Q = CV$) to change voltage, thus for a fixed drive strength, transitions become slower.
- Poor slew rates prevent reliable logic capture.

Signal transition should remain within a **permissible range** to ensure reliable operation throughout the design.

Placement Optimization

Optimized placement aims to minimize total wire length and improve signal transmission by:

- Clustering related cells.
- Placing critical elements to minimize direct distances to their respective ports or connected cells.
- Reducing the need for large numbers of repeaters or buffers, which consume additional area and resources.

This process is iterative and involves estimating wire lengths, evaluating resulting capacitances and resistances, and predicting performance impacts before finalizing layout.

Key Points

- **Strategic cell placement** is critical for signal integrity and reduces the need for additional buffers/repeaters.
 - Wire **capacitance and resistance** are primary factors in determining maximum reliable wire lengths.
 - **Repeaters/buffers** are essential for maintaining signal quality over large distances but consume extra silicon area.
 - **Slew rate** is determined by load capacitance and affects the ability to transition between logic levels reliably.
 - *Optimized placement* reduces unnecessary interconnect length, enhances performance, and manages power/area tradeoffs.
 - Evaluating the necessity of repeaters and analyzing transition integrity forms part of broader **timing analysis**, which considers all signal paths and delays in detail.
-

Final placement optimization

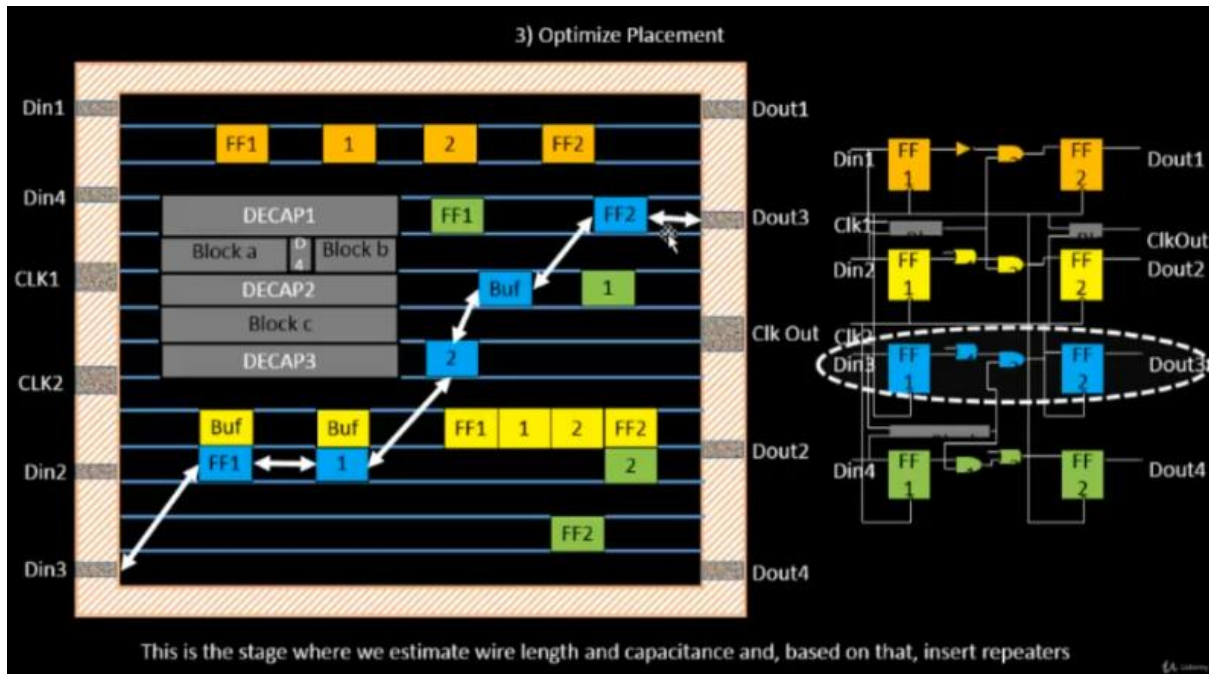
This documentation covers the theoretical foundation and key concepts related to placement optimization and ideal-clock timing analysis in digital circuit design, particularly as applied in open-source hardware and FPGA workflows. Topics include the motivation for buffer insertion, abutment for high-frequency paths, constraints imposed by physical layout, and the procedure and rationale for timing analysis with ideal clocks.

Core Concepts

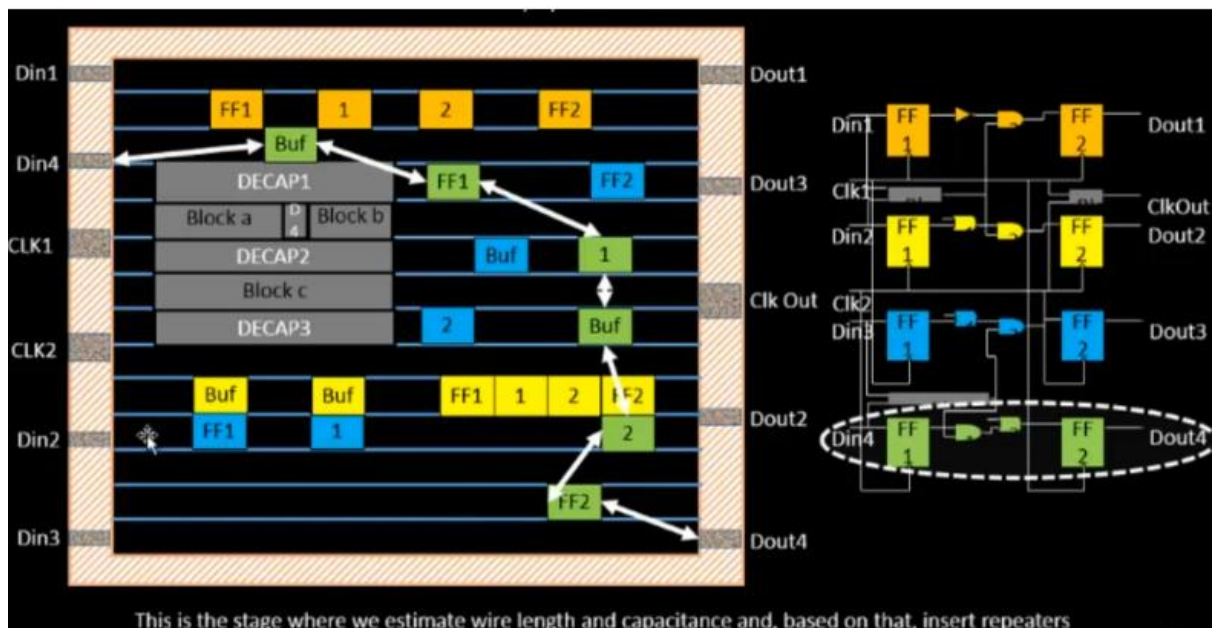
Placement Optimization

- **Placement** is the process of determining the optimal physical locations for logic elements (such as buffers, flip-flops, and gates) on a hardware chip to satisfy design constraints and performance objectives.
- Goals include minimizing wire lengths, signal delays, power consumption, and avoiding congestion, while satisfying design constraints like proximity to inputs/outputs, resource availability, and logical connectivity.
- **Buffer Insertion:** Buffers (or repeaters) are strategically placed to recondition signals that must travel long distances. They ensure the signal at the destination matches the original in timing and waveform integrity, crucial for reliable operation in distributed circuits operating at high frequency.

- Proper buffer placement is dictated by signal transition requirements and the max allowable wire length for a reliable transmission, often determined through data transition analysis.



- Abutment:** High-speed logic blocks are physically placed very close to each other to minimize interconnect delay—sometimes reducing it nearly to zero. This technique is particularly deployed in sections of the design that must operate at very high frequencies (e.g., 2 GHz), where even small delays can affect overall timing.



- **Physical Constraints:** Placement is affected by other hardware resources, such as decoupling capacitors and existing blocks, which can prevent optimal buffer placement in constrained regions. Routing and buffer strategies must adapt to these constraints.
- **Crisscross and Layer Conflicts:** Signal crossings and congestion are resolved in later routing stages, often by assigning connections to different metal layers (e.g., Metal 3 vs. Metal 4).
- **Timing Analysis with Ideal Clocks**
- After placement optimization, a **timing analysis** is performed with the assumption of *ideal clocks*—meaning the clock signal reaches all relevant elements with zero delay.
- This idealized analysis isolates the data path timing and helps identify whether the placement and buffer insertion will satisfy the required frequency specifications before real, non-ideal clocks are considered.
- *Setup timing analysis* checks that data can propagate through the placed logic within the clock period. If setup timing fails in ideal conditions, adjustments to placement are needed before advancing to more detailed (and less forgiving) timing analyses.
- Notably, *hold analysis* is not meaningful under ideal clocks, as all flip-flops are triggered simultaneously with zero skew or latency.
- **Key Points**
- Placement optimization directly impacts the performance and reliability of digital circuits by configuring physical locations of all elements to mitigate delays and maximize throughput.
- Buffer insertion is crucial when signals must traverse long wires; correct placement ensures high-fidelity signal reproduction, essential for synchronous operation at high frequencies.
- Adjacency (abutment) of fast-path logic blocks eliminates unnecessary wire delays and is a common strategy for segments of a circuit operating at the highest speeds.
- Physical layout often imposes constraints, such as regions reserved for capacitors or other blocks, necessitating route adjustments or alternative buffer placements.
- Layered routing is used to resolve crisscrossed or overlapping connections in complex layouts, ensuring logical connectivity without shorts or resource conflicts.
- Timing analysis with ideal clocks is a vital verification step post-placement, providing an early check on the feasibility of meeting timing specifications and informing further optimization if necessary.
- Setup timing is expressed as:

$$T_{setup} \leq T_{clock} - (T_{comb} + T_{routing} + T_{clock_q})$$

Where:

T_{setup} = Setup time requirement

T_{clock} = Clock period

T_{comb} = Combinational logic delay

$T_{routing}$ = Routing (wire/buffer) delay

T_{clock_q} = Flip-flop clock-to-q delay

- Meeting setup timing in ideal clocks is a prerequisite; failure here indicates the need for placement changes before more nuanced routing and real-clock analysis.
- Optimizing at this stage reduces complexity in subsequent (more computationally intensive) placement and routing phases as well as the risk of failure due to unmet timing or congestion.
- **Need for libraries and characterization**
- This document outlines the **theoretical foundations** of integrated circuit (IC) design flow, with a focus on how each stage interacts with library characterization. The stages include logic synthesis, floorplanning, placement, clock tree synthesis, routing, and static timing analysis, culminating in the necessity for comprehensive gate characterization. Each stage relies on accurate cell models to achieve targeted functionality, timing, and manufacturability.

-

-

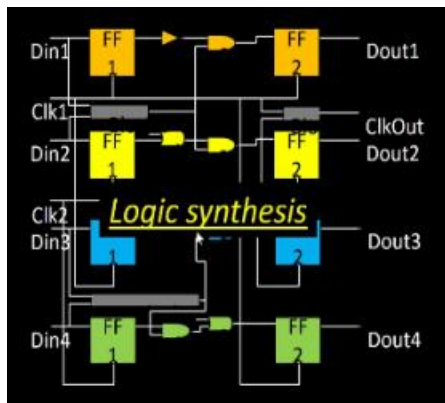
- **Core Concepts**

- **IC Design Flow Overview**

- The **IC design flow** is a stepwise methodology used to convert a high-level functional description into a manufacturable circuit. It integrates hardware description languages, logical arrangement of gates, and physical placement, all orchestrated to meet specific design goals (functionality, area, speed, power). Library characterization underpins each step by providing the necessary cell models and parameters that design tools use.

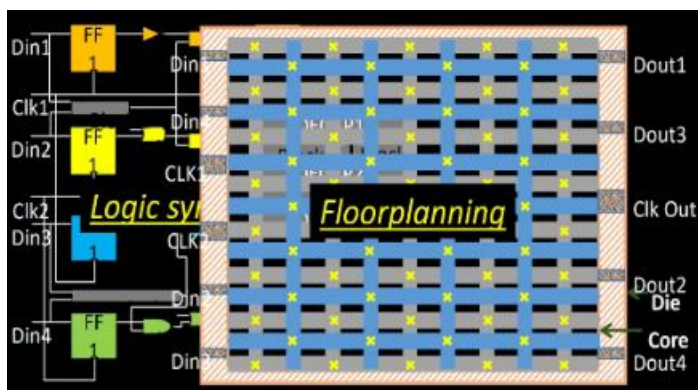
- **Logic Synthesis**

- **Logic synthesis** transforms a high-level functionality (typically specified in RTL) into an arrangement of gates—such as AND, OR, buffers, and flip-flops—that implements the desired hardware. The synthesized netlist is a gate-level representation optimized for area, speed, and power, guided by the characteristics of the cell library.



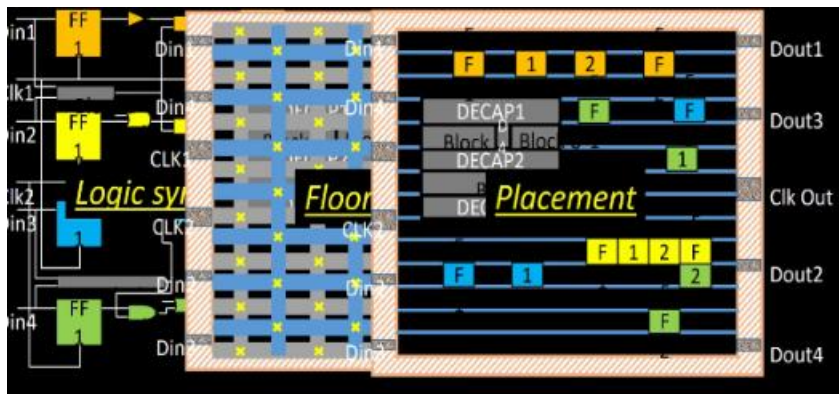
Floorplanning

Floorplanning imports the gate-level netlist and translates it into a spatial layout. Decisions regarding the width and height of the core and die are based on the quantity and sizes of gates. The shaping and area allocation directly depend on the gate properties defined during characterization, impacting all downstream physical design phases.



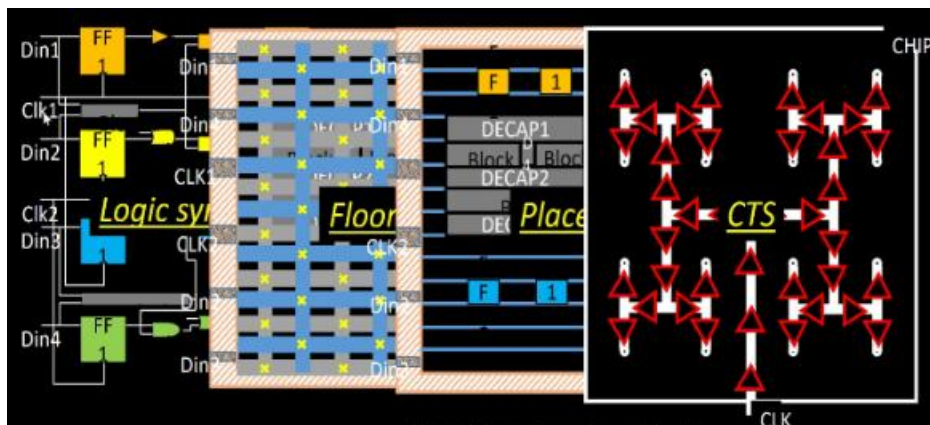
Placement

Placement arranges individual logic cells onto the chip to optimize for timing and connectivity. Logical proximity—dictated by netlist connections—drives physical closeness, aiming to minimize path delays (including setup and hold time violations). Placement strategies may adjust cell distances and introduce buffers to maintain design targets, all relying on accurate cell timing models.



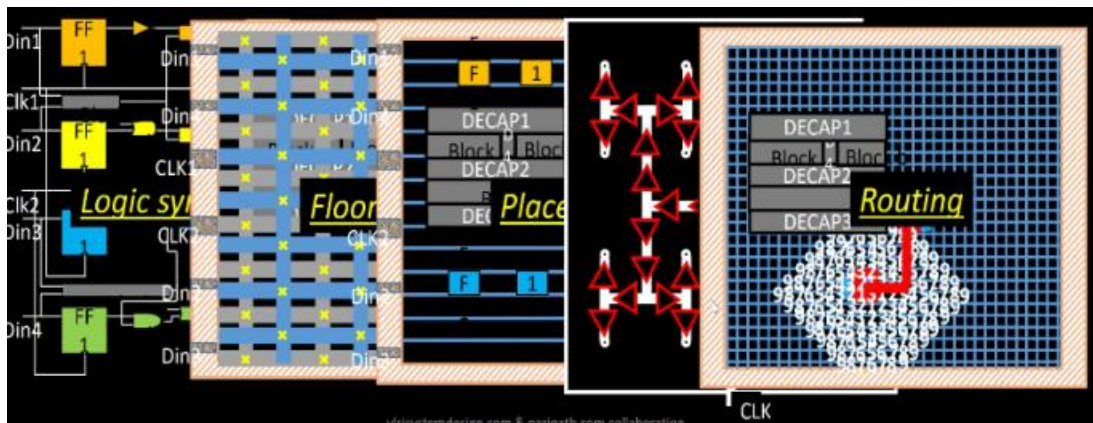
Clock Tree Synthesis

Clock tree synthesis ensures that the clock signal reaches all destinations (clock endpoints) simultaneously, achieving minimal skew. Buffer cells are inserted so every point receives the clock with equal rise/fall times. This process is critically dependent on the timing and electrical characteristics provided by the library for each buffer and flip-flop.



Routing

Routing connects cell terminals according to logical and physical constraints. Techniques like maze routing consider cell properties, such as output drive strength and input capacitance, to maintain signal integrity and timing. The routing step must honor cell specifications laid out in the characterization phase.

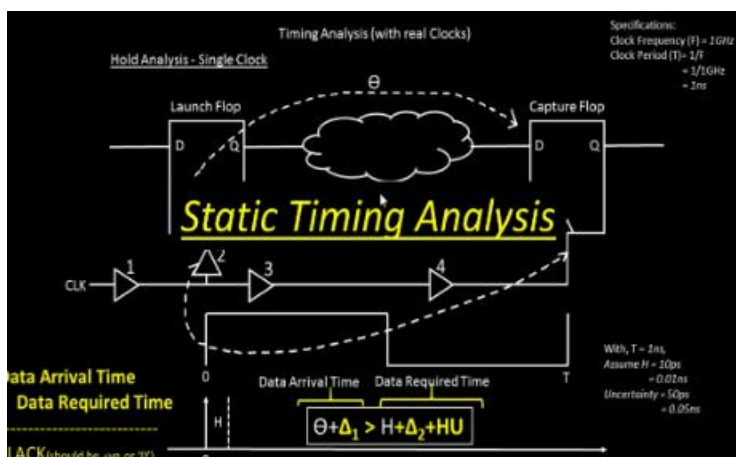


Static Timing Analysis

Static timing analysis (STA) evaluates timing paths to identify potential setup and hold violations and calculates the maximum achievable frequency. Core equations include:

- Data Arrival Time:
 - $T_{arrival} = T_{launch} + T_{delay}$
- Setup and Hold Requirements:
 - $T_{setup} \leq T_{clk} - T_{arrival}$
 - $T_{hold} \geq T_{arrival} - T_{clk}$

The accuracy of STA is fundamentally tied to the timing data characterized for each cell.



Library Characterization

At every step, the design process references **standard cells** (AND, OR, buffer, inverter, flip-flop, latch) from the cell library. These cells must be characterized for timing, electrical, and physical properties so EDA tools can interpret their behavior accurately.

Without detailed models, the design tools cannot guarantee proper functionality or timing.

One Common Thing across all stages
"GATES or Cells"

Library

- AND gate
- OR gate
- BUFFER
- INVERTER
- DFF
- LATCH
- ICG
-

How do we design, characterize or model these cells?

vissystemdesign.com & partpat.com collaboration

Key Points

- Each stage of IC design flow—logic synthesis, floorplanning, placement, clock tree synthesis, routing, and timing analysis—relies on accurate and comprehensive library characterization.
- A **cell library** consists of gate definitions and their associated models, representing the fundamental building blocks used throughout the design flow.
- **Library characterization** is the process of modeling the gates so that EDA tools can understand their functional, timing, and physical behavior.
- Timing equations (e.g., setup time, hold time) are used to ensure reliable operation:
 - Typical setup time analysis: $T_{setup} = T_{clk} - T_{arrival}$
 - STA checks for violations that could affect performance.
- Physical design stages (floorplanning, placement, routing) depend on accurate area and delay numbers from characterization, directly affecting manufacturability and performance.

Congestion aware placement using RePLAcE

This lab focuses on the **placement stage** in digital design using open-source hardware tools. The main aim is to perform placement with an emphasis on congestion reduction rather than timing optimization. Placement consists of two distinct phases: **global**

placement and **detailed placement**. In this lab, you will use relevant commands and tools to optimize wire length, legalize the placement of standard cells, and verify results visually. The process includes observing parameters such as **half perimeter wire length (HPWL)** and **overflow**, checking standard cell alignment, and viewing the design post-placement using visualization tools.

Lab Steps

1. Run Placement

- Placement occurs in two stages:
 - **Global Placement:** Coarse positioning aimed at reducing overall wire length.
 - **Detailed Placement:** Legalizes cell positions, ensuring no overlaps and proper alignment within standard cell rows.
- Start the placement run:
- `run_placement`

```
[WARNING PSM-0030] Vsrc location at (425.520um, 430.880um) and size =10.000um, is not local to
a power stripe. Moving to closest stripe at (426.600um, 410.240um).
[WARNING PSM-0030] Vsrc location at (565.520um, 430.880um) and size =10.000um, is not local to
a power stripe. Moving to closest stripe at (567.000um, 410.240um).
[WARNING PSM-0030] Vsrc location at (285.520um, 570.880um) and size =10.000um, is not local to
a power stripe. Moving to closest stripe at (286.200um, 563.420um).
[WARNING PSM-0030] Vsrc location at (425.520um, 570.880um) and size =10.000um, is not local to
a power stripe. Moving to closest stripe at (426.600um, 563.420um).
[WARNING PSM-0030] Vsrc location at (565.520um, 570.880um) and size =10.000um, is not local to
a power stripe. Moving to closest stripe at (567.000um, 563.420um).
[INFO PSM-0031] Number of nodes on net VGND = 19223.
[INFO PSM-0037] G matrix created successfully.
[INFO PSM-0040] Connection between all PDN nodes established in net VGND.
[INFO]: PDN generation was successful.
[INFO]: Changing layout from /openLANE_flow/designs/plcorv32a/runs/30-10_15-39/results/floorplan
picorv32a.floorplan.def to /openLANE_flow/designs/plcorv32a/runs/30-10_15-39/tmp/floorplan
def
1
% run_placement
```



```

total instances      21699
multi row instances   0
fixed instances      6354
nets                15449
design area          420473.3 u^2
fixed area           9141.3 u^2
movable area         147800.5 u^2
utilization          36 %
utilization padded   55 %
rows                 238
row height           2.7 u

Placement Analysis
-----
total displacement    0.0 u
average displacement  0.0 u
max displacement      0.0 u
original HPWL         766080.0 u
legalized HPWL        779196.5 u
delta HPWL            2 %

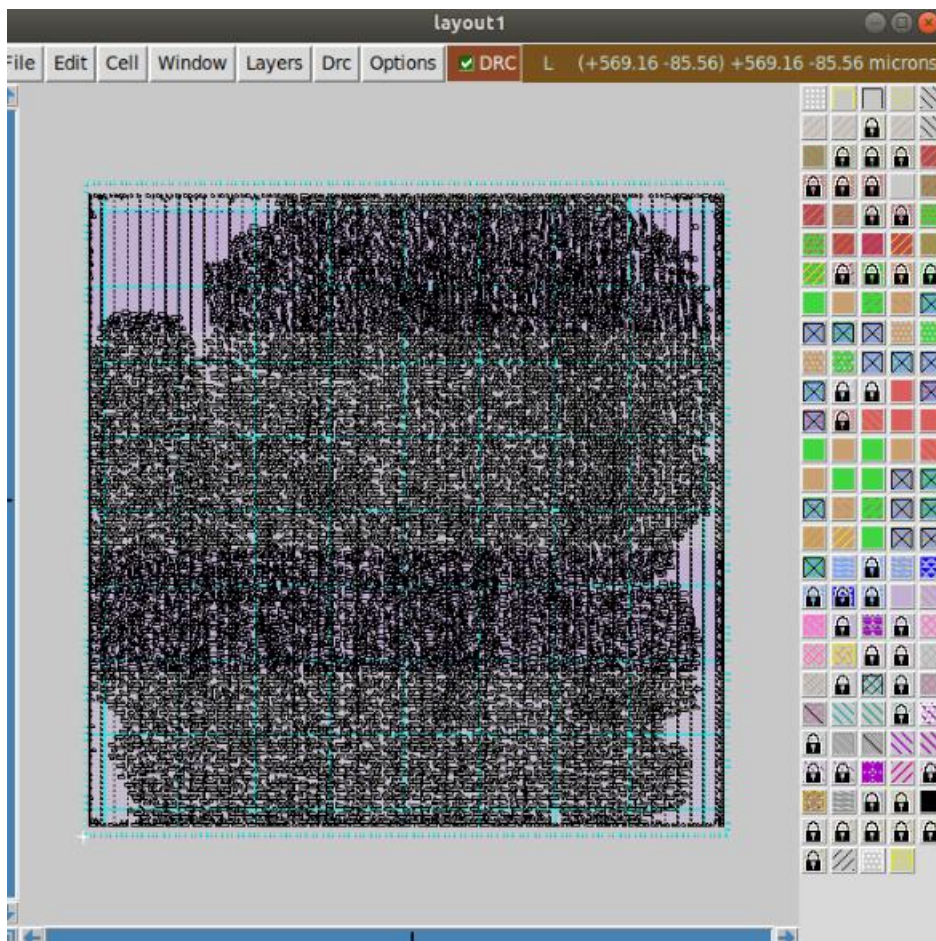
[INFO DPL-0020] Mirrored 6193 instances
[INFO DPL-0021] HPWL before      779196.5 u
[INFO DPL-0022] HPWL after      766080.0 u
[INFO DPL-0023] HPWL delta      -1.7 %
[INFO]: Changing layout from /openLANE_flow/designs/plcorv32a/runs/30-10_15-39/tmp/placement/8-re
sizer.def to /openLANE_flow/designs/plcorv32a/runs/30-10_15-39/results/placement/plcorv32a.placem
ent.def
[INFO]: Changing layout from /openLANE_flow/designs/plcorv32a/runs/30-10_15-39/results/placement/
plcorv32a.placement.def to /openLANE_flow/designs/plcorv32a/runs/30-10_15-39/results/placement/pl
corv32a.placement.def
[INFO]: Taking a Screenshot of the Layout Using Klayout...
[INFO]: current step index: 12
Using Techfile: /home/vsduser/Desktop/work/tools/openlane_working_dir/pdks/sky130A/libs.tech/klay
out/sky130A.lyt
Using layout file: /openLANE_flow/designs/plcorv32a/runs/30-10_15-39/results/placement/plcorv32a.

```

2.

-
- **Monitor Algorithm and Output Metrics**
 - During **global placement**, the key metric is **HPWL (half perimeter wire length)**. The goal is to minimize this value for better placement quality.
 - Another critical parameter is **overflow**. Convergence is indicated when the overflow value decreases through iterations.
 - # Example output
 - HPWL: 150000
 - Overflow: 1200
 - HPWL: 140000
 - Overflow: 800
 - ...
- **Verify Legalization After Placement**

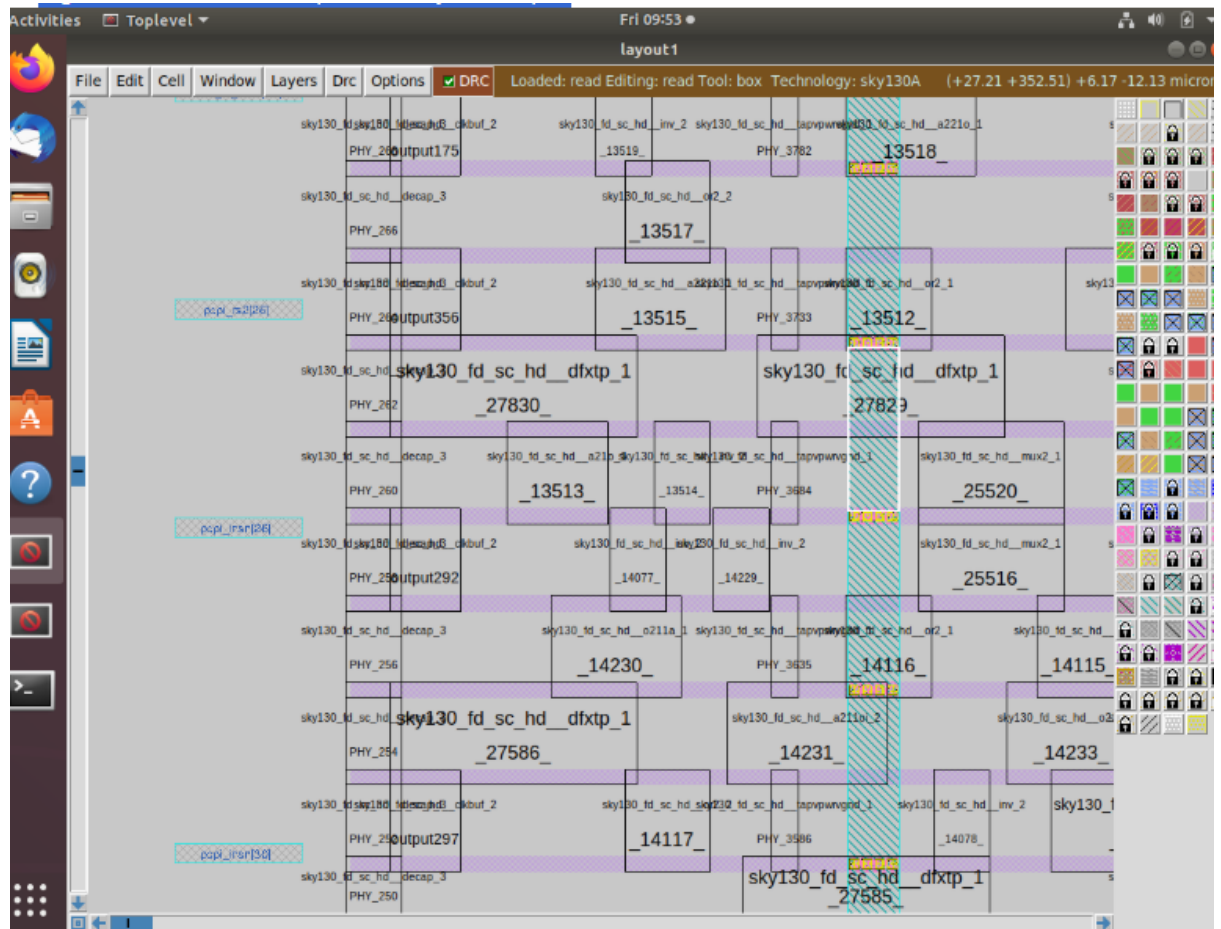
- In **detailed placement**, the process ensures that all standard cells are:
 - Properly aligned inside standard cell rows.
 - Have no overlaps.
 - Legalized from physical and timing standpoints.
- **Visualize Placement with Magic Tool**
 - Once placement is complete, view the resulting layout:
 - magic -T
 /home/vsduser/Desktop/work/tools/openlane_working_dir/pdks/sky130A/libs.tech/magic/sky130A.tech lef read ../../tmp/merged.lef
 def read picorv32a.placement.def &

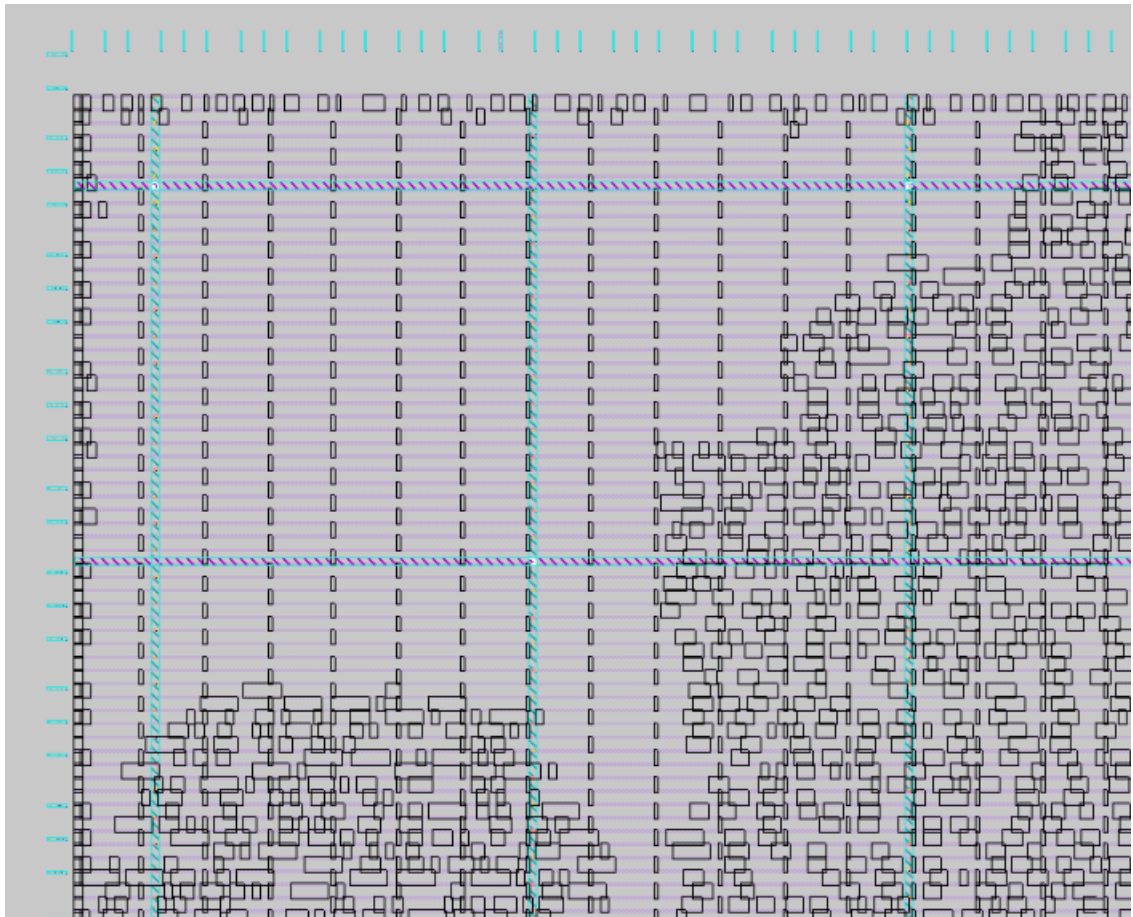


Use Magic's visualization features to inspect:

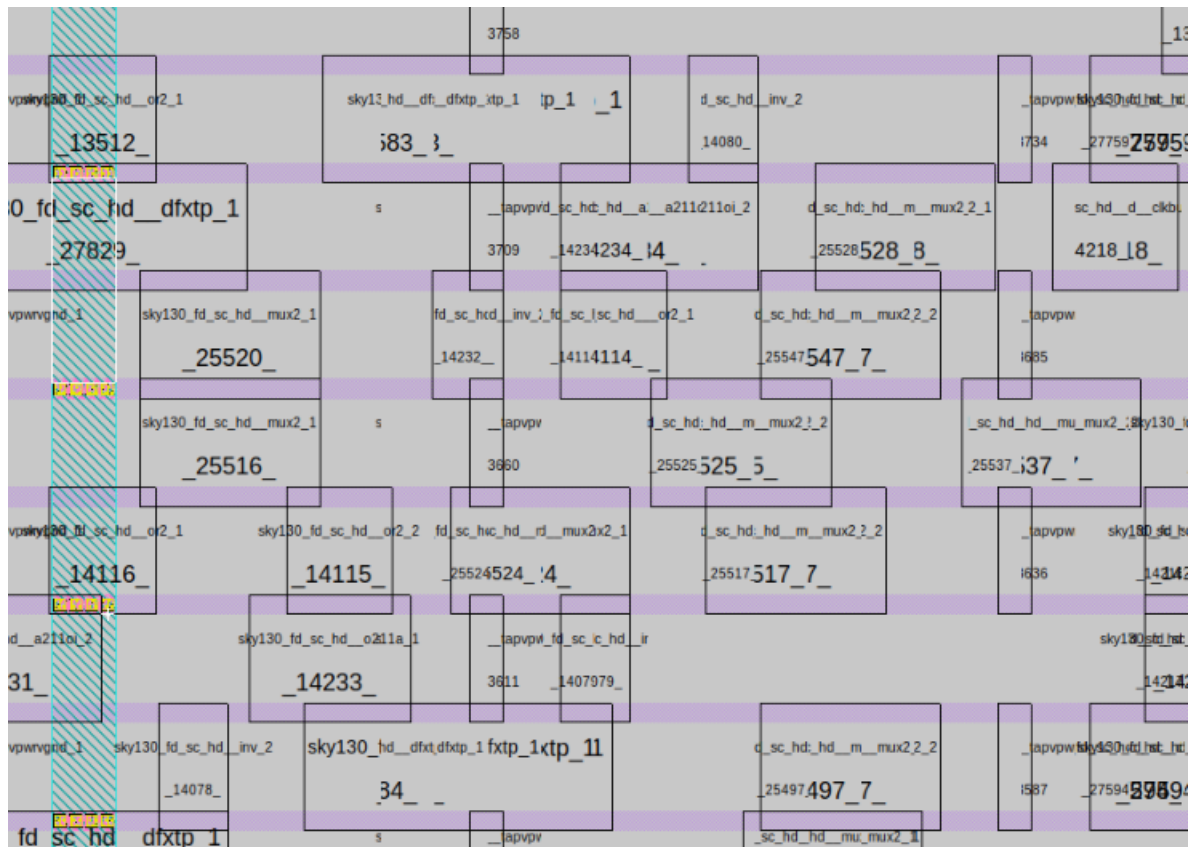
- Centralized standard cell clusters

- Correct placement of single cells (often observed in the lower left corner after floorplan)
- Alignment of TAP cells and IO pads as set by the floorplan





-
- Zoom in to check detailed placement of standard cells within their rows.



-

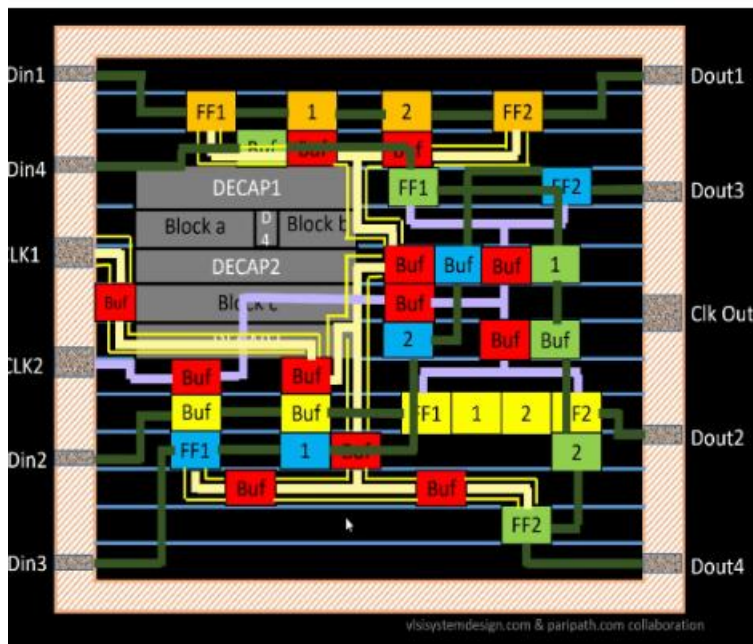
- **Review Standard Cell Placement and Boundary Conditions**
 - Confirm:
 - **No DRC (Design Rule Check) violations**
 - Standard cells are correctly placed within rows and boundaries
 - Decap cells are at the boundaries, TAP cells and IO pads are properly positioned per floorplan output
- **Save the Design State**
 - Save visualization or DEF file for reference.
 - save <output-name>.def
- **Next Steps**
 - Note: The **Power Distribution Network (PDN)** is not yet created. In this flow, PDN generation happens after placement and CTS, before routing.

Cell design and characterization flows

Inputs for cell design flow

This documentation provides a comprehensive theoretical overview of the cell design flow in integrated circuit (IC) development, with an emphasis on the concept and role of standard cells, standard cell libraries, and the foundational elements provided by foundries. It gives a high-level understanding of how standard cells are prepared for use in digital design, the types of information housed in cell libraries, and the key inputs required to design a basic cell such as an inverter.

-

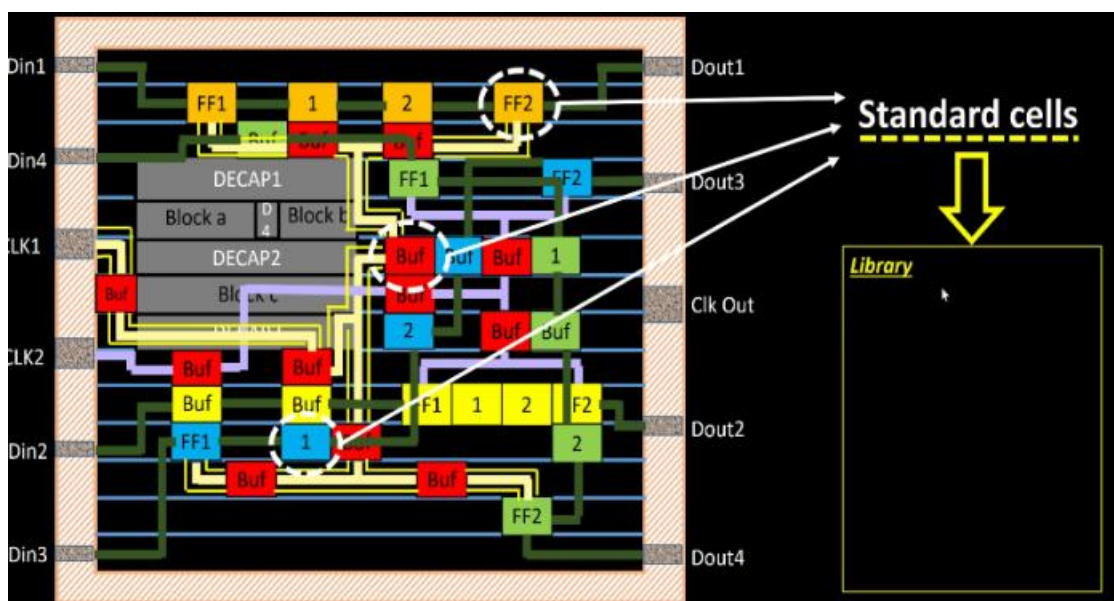


Core Concepts

Standard Cells

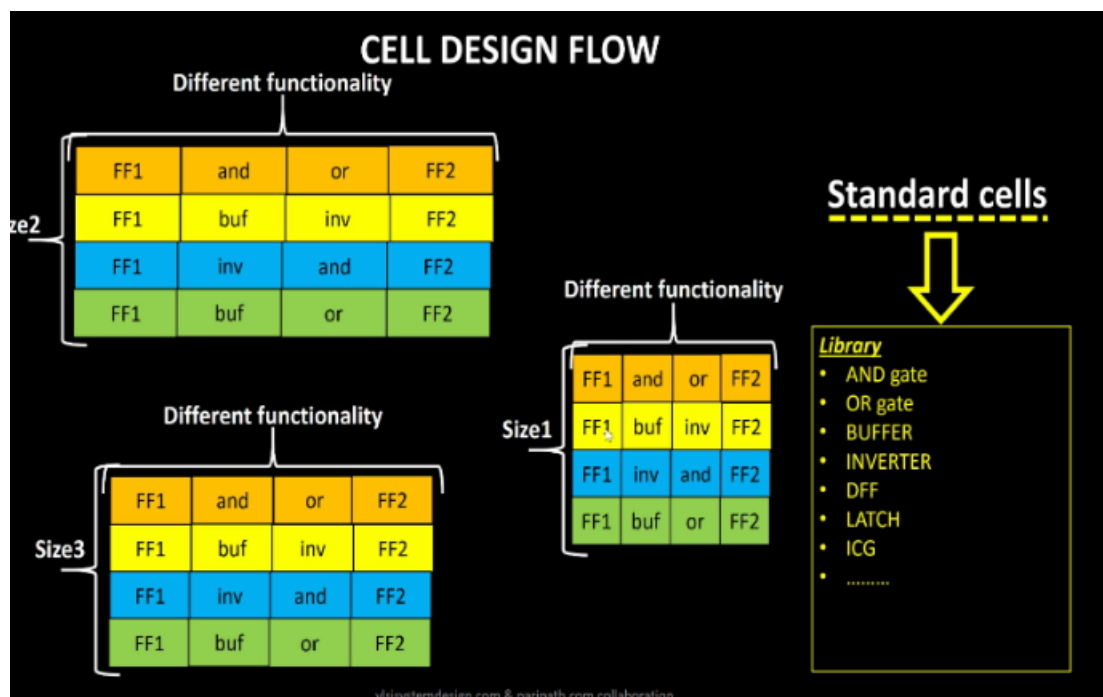
- **Standard cells** are basic logic gates or functional blocks (e.g., AND gates, OR gates, inverters, buffers, flip-flops) used repeatedly in digital IC design.
- These cells are pre-designed, characterized, and stored in a **library** for reuse in placing and routing digital designs.

-



Standard Cell Library

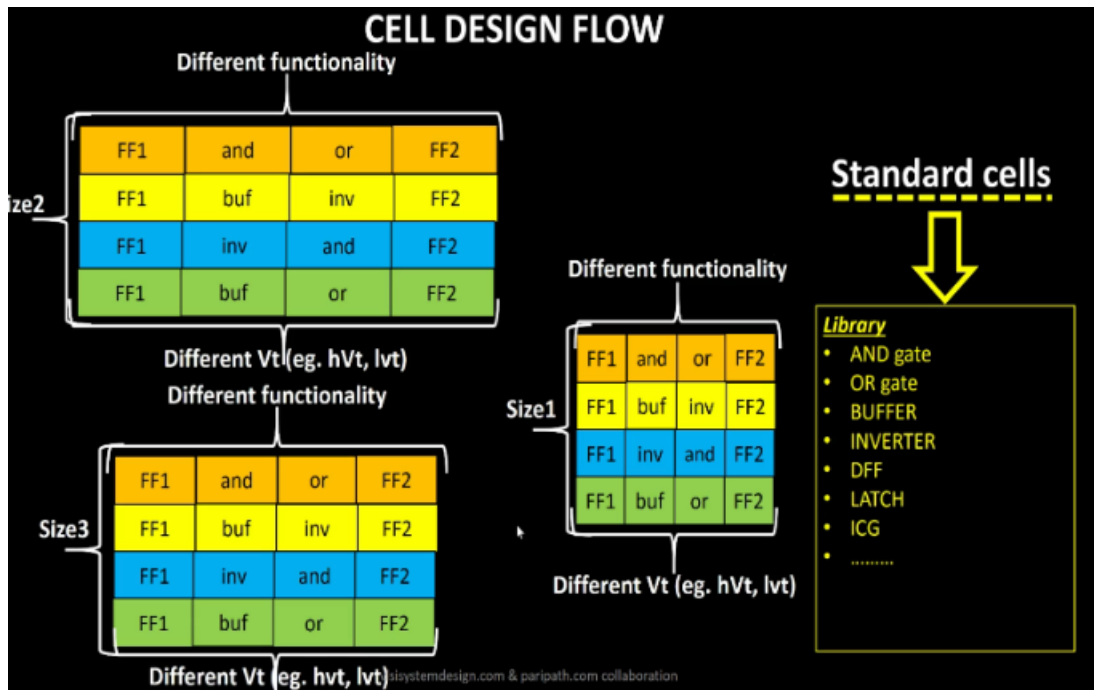
- The **library** is a foundational component in the IC design flow, analogous to a bookshelf containing a wide variety of books—here, it contains standard cells and possibly other blocks such as decoupling capacitors, macros, and IPs.
- Libraries contain cells with:
 - **Different functionalities:** e.g., logic gates, storage elements.
 - **Different sizes:** This refers to *drive strengths*, where a physically larger cell can source/sink more current, hence better drive longer or heavily loaded wires.
 - **Different threshold voltages (V_t):** Cells that differ in V_t are optimized for different performance/leakage trade-offs. For example, a cell with lower V_t switches faster but may leak more.



Physical and Electrical Characteristics

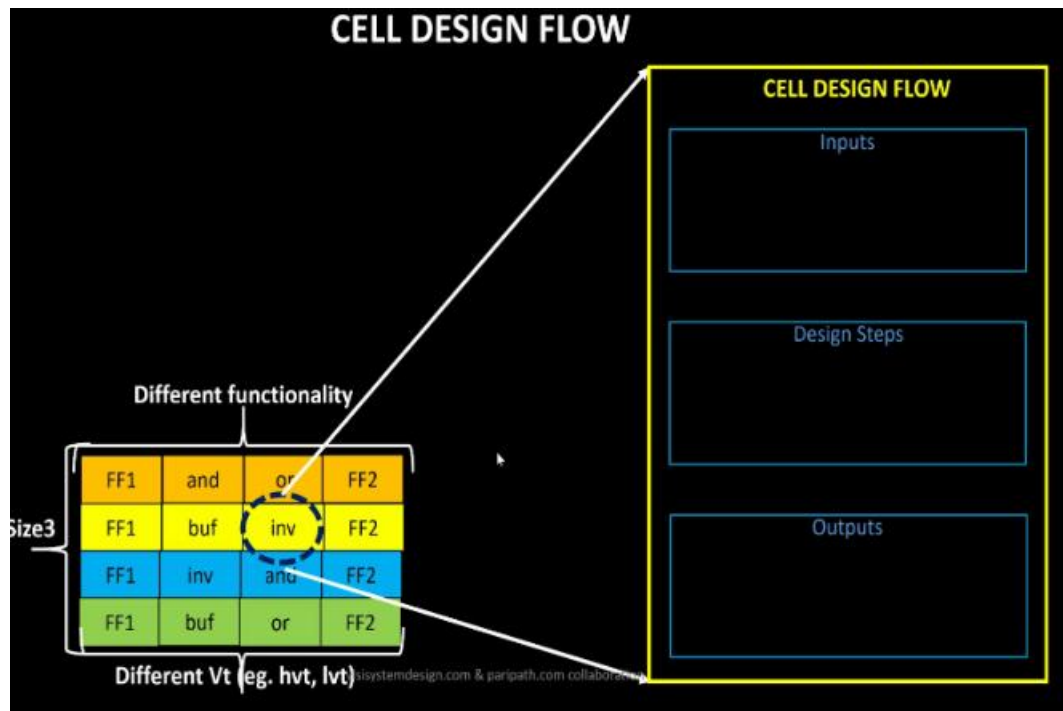
- **Physical size:** Affects drive strength, generally with larger devices providing greater output current.

- **Threshold voltage (V_t):** Influences both speed and leakage power. Lower V_t means faster switching but more leakage; higher V_t means slower switching but less leakage.
- Libraries may contain otherwise identical cells that differ only in V_t or drive strength, supporting flexible design optimization.



- **Cell Representation for EDA Tools**
- Each cell is represented not just by its schematic symbol but also by:
 - **Physical shape:** For placement/layout.
 - **Timing characteristics:** Delay, setup/hold times.
 - **Drive strength and load capabilities.**
 - **Power consumption data:** Both static and dynamic.
- EDA (Electronic Design Automation) tools use these attributes to optimize the placement, timing closure, and power characteristics of the final chip.
- **Cell Design Flow Overview**
- The cell design flow can be divided into three main categories:
 - **Inputs:** Requirements and constraints provided by the foundry and users.
 - **Cell Design:** Application of rules and models to create a functional cell.
 - **Outputs:** Characterized models used by EDA tools.

- This explanation focuses on the **inputs** involved in designing a cell such as an inverter.



Inputs to Cell Design

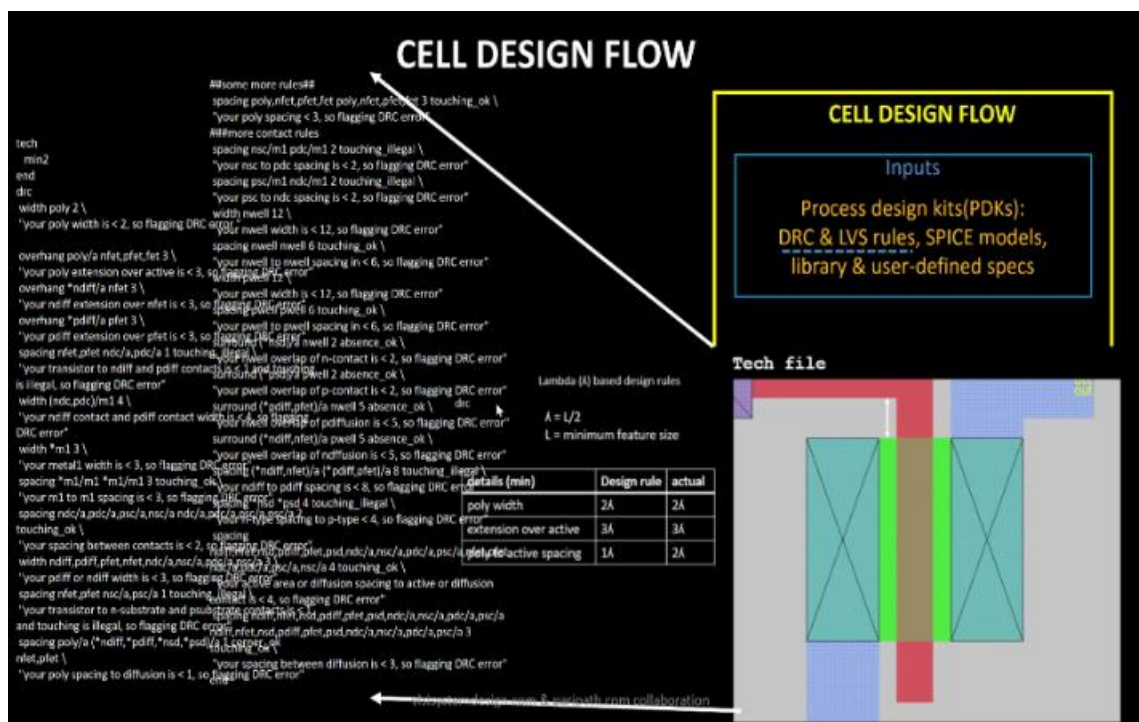
- **Process Design Kits (PDK):**
 - Provided by the semiconductor foundry.
 - Includes:
 - **Design Rule Check (DRC) and Layout Versus Schematic (LVS) rules:** Dictate layout restrictions (e.g., minimum/maximum widths, spacing, overlaps) to ensure manufacturability and correct function.
 - Required for custom layout and standard cell development.
- **SPICE Model Parameters:**
 - Also from the foundry.
 - Provide electrical models for transistor behavior used in simulation and characterization.
 - Include key physical/electrical parameters, such as:
 - **Oxide thickness** (t_{ox}), e.g., $t_{ox} = 5.8, nm$
 - **Nominal threshold voltages** (V_t), e.g., $V_t = 0.5, V$
 - **Capacitance values:** (e.g., C_{ox} , C_{gs} , C_{gd} , etc.)
 - **Substrate and process-specific constants**

Example SPICE equations:

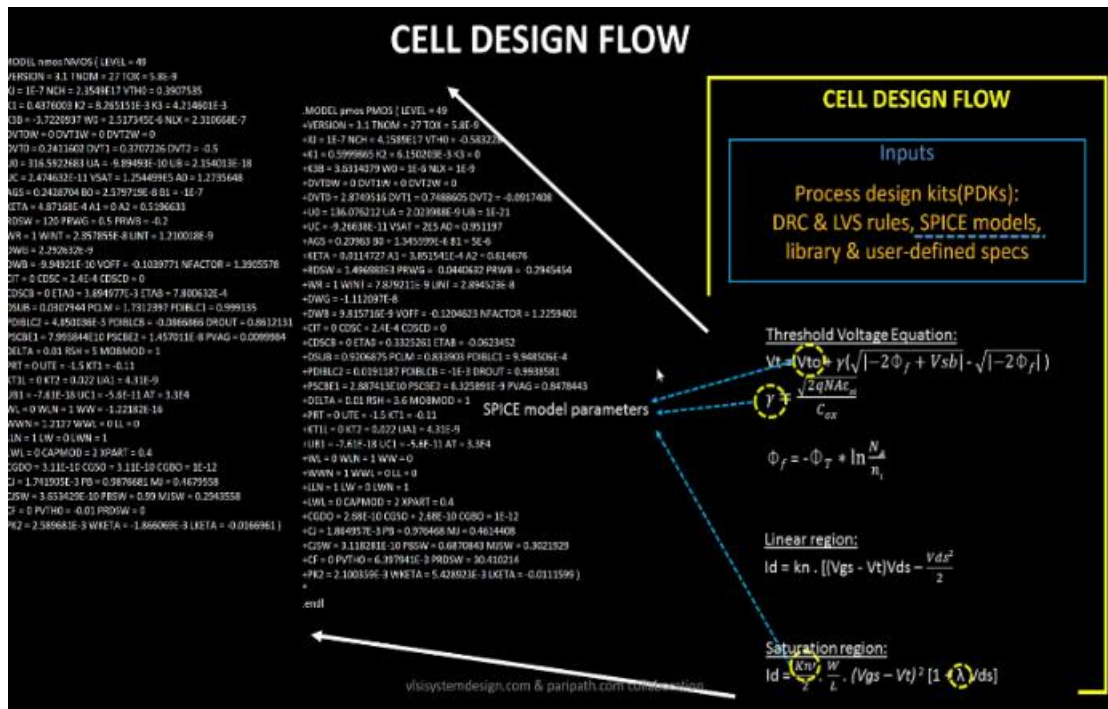
- Threshold voltage:
 - $V_T = V_{T0} + \gamma(\sqrt{|2\phi_B + V_{SB}|} - \sqrt{|2\phi_B|})$
- Oxide capacitance:
 - $C_{ox} = \frac{\epsilon_{ox}}{t_{ox}}$

Library and User-Defined Specifications:

- Custom requirements, functional specifications, and any additional constraints or targets for the cell.



- Design Rule Examples**
- Poly width minimum: 2λ**
- Poly extension over active area: 3λ**
- Poly to active spacing: 1λ**
- These rules help ensure manufacturability and reliable device operation. There are typically thousands of such rules in a full process design kit.



Key Points

- **Standard cells and libraries** are fundamental building blocks for digital IC design, enabling modular design reuse.
- Libraries contain functionally diverse, size-varied, and electrically distinct versions of each standard cell for design flexibility.
- **Drive strength** and **threshold voltage (V_t)** are critical characteristics used to tailor standard cell performance to the needs of the chip.
- The **cell design flow** relies heavily on detailed inputs from the foundry:
- **Physical design/spacing rules (DRC, LVS)**
- **SPICE models and parameters** for accurate electrical characterization.
- Each cell must be thoroughly described for placement, timing, drive/power, and verified against foundry-provided rules and models.
- The process ensures that custom or standard cells are manufacturable, meet performance targets, and are compatible with automated design tools.

Circuit design step

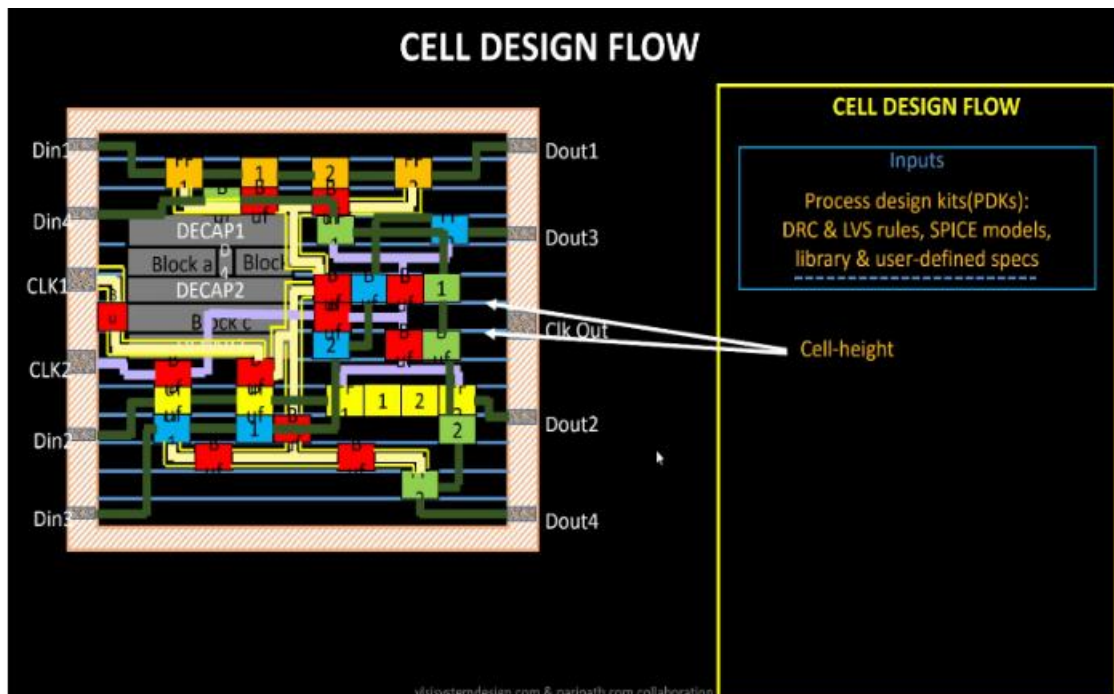
- This document explains the theoretical foundations behind user-defined specifications in standard cell library development for open-source hardware design. Key concepts discussed include the relationship between physical

design parameters and circuit performance requirements, as well as the primary design stages of standard cell creation: circuit design, layout design, and characterization.

- **Core Concepts**

- **User-Defined Specifications in Standard Cell Libraries**

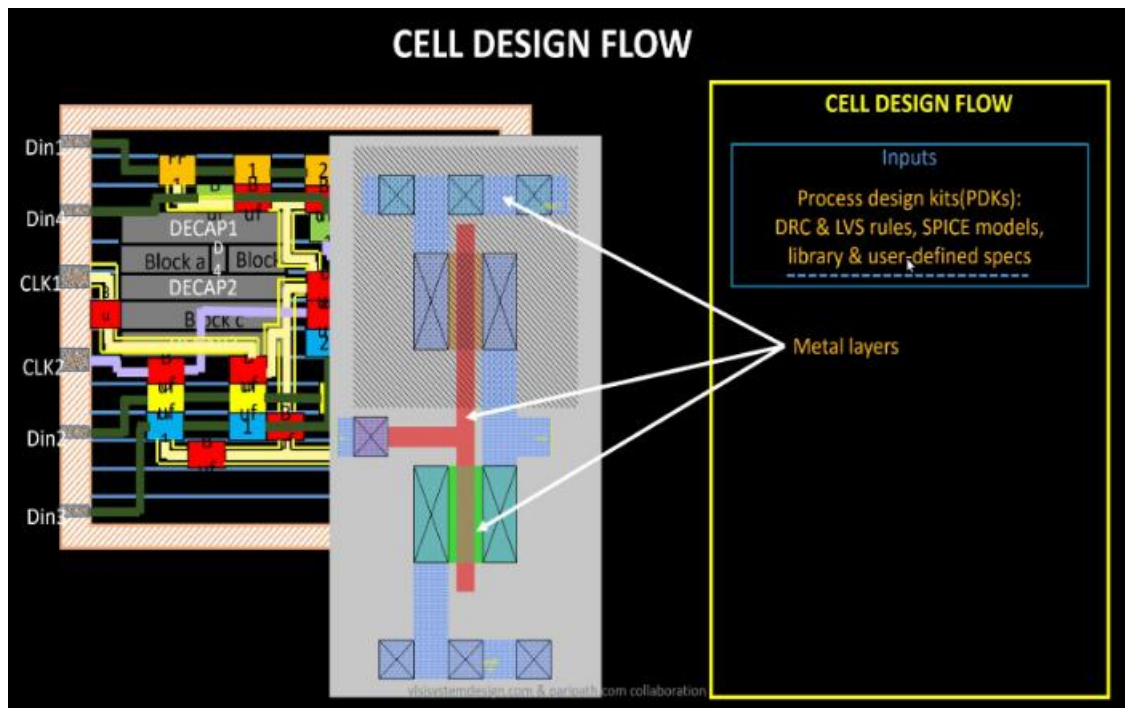
- User-defined specifications drive the development of standard cell libraries by dictating the physical and electrical properties cells must meet to integrate successfully into semiconductor design environments.
- **Cell Height** is determined by the separation between the power rail (VDD) and the ground rail (VSS). The library developer is responsible for maintaining this height based on the provided specifications.



-
-
- **Cell Width** is related to the drive strength required by the user. Drive strength indicates a cell's ability to drive electrical load.
 - Low drive strength (Drive Strength = 1) makes a cell less capable of controlling large wire loads.
 - High drive strength (e.g., Drive Strength = 10) allows the cell to drive longer or heavily loaded wires.
- **Supply Voltage** is specified at the top level. The standard cell must be designed to operate at this specified voltage, influencing transistor sizing and

ensuring correct noise margins. Typical values might include $V_{dd} = 1.2V$, and the threshold voltage needs to be considered for noise immunity.

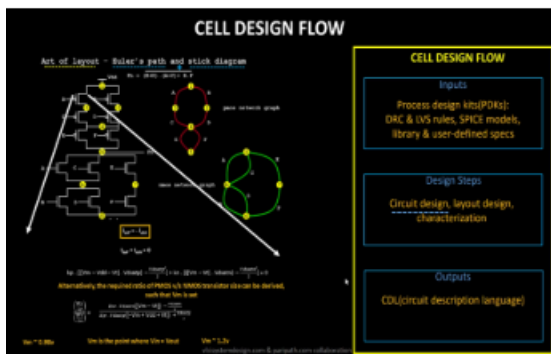
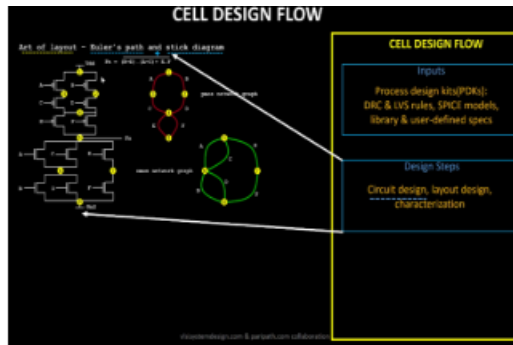
- **Metal Layer Requirements** define which layers (e.g., Poly, Metal1, Metal3) are used for different cell components (e.g., power, ground, signal contacts). Specific design constraints can require certain connections or contacts to occupy particular metal layers for routing or process compatibility.



Key Points

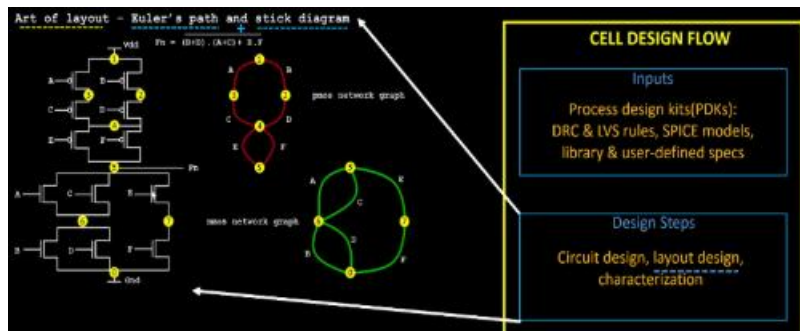
- **Spec Driven Design:** The library developer must carefully interpret and implement all user-defined specifications so that cells promote successful integration through every stage of the chip design flow.
- **Design Process Stages:**
 - **Circuit Design** involves two core steps:
 - **Functional Implementation:** Defining the logic function the cell will implement.
 - **Transistor Sizing:** Modelling PMOS and NMOS transistors, primarily optimizing width-to-length (W/L) ratios to meet electrical specifications like drive strength and switching threshold.

- Example relationship: $W_p = 2.5 \times W_n$, where W_p and W_n are widths for PMOS and NMOS, respectively.
- The drain current requirement must be met:
- $I_{D,p} + I_{D,n} = 0$
- The switching threshold voltage is set to a target value (e.g., $V_{th} = 1.2, V$) based on the application.



- **Layout design step**
- This section elaborates on the theoretical process of digital layout design for standard cells, emphasizing the translation from circuit function to physical layout, and the requirements imposed by both foundry rules and user specifications. Key steps include network graph derivation, Euler path extraction, stick diagram generation, and the final production of manufacturable layouts adhering to design rule checks (DRC). Parasitic extraction and cell characterization follow to ensure timing, noise, and power compliance.
- **Core Concepts**
- **Function-to-Transistor Network Mapping**
- The initial theoretical step involves implementing a specified logic function using connections of PMOS and NMOS transistors. Each given function corresponds to a specific transistor configuration, forming the foundation for all later physical abstractions.

•

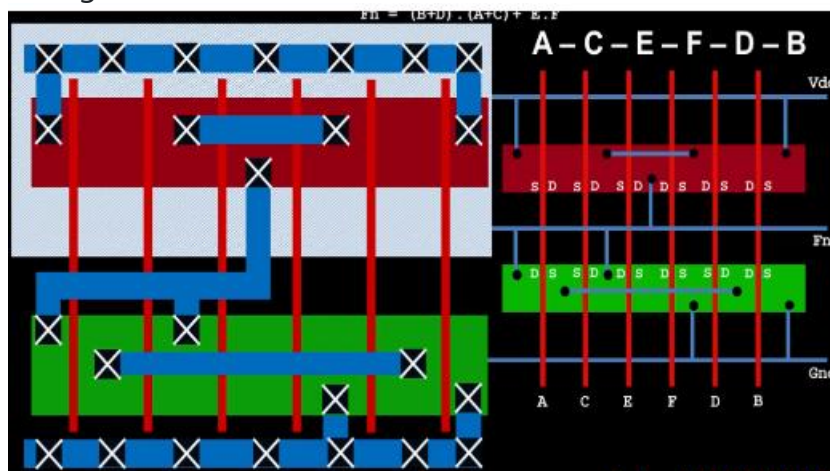


Stick Diagram Generation

With the Euler path established, a stick diagram is produced. This abstract representation shows the sequence of gate and diffusion connections as simplified lines and contacts, encoding the order derived from the Euler path.

- Input sequence (e.g., ACEF, DNB) guides polygate placement and interconnections.
- Sources, drains, and gates are abstracted, emphasizing topological relations over geometric detail.

•



Physical Layout Adhering to DRC and Specifications

The stick diagram is translated into a detailed physical layout, observing strict Design Rule Check (DRC) guidelines provided by the semiconductor foundry. User-defined specifications such as gate length, contact locations, and pin placements are incorporated at this stage.

- Multiple metal layers (metal 1, metal 2, etc.) and polysilicon are organized to match functional and manufacturing requirements.
- The cell's physical dimensions (width, height) emerge from these constraints and optimization strategies.

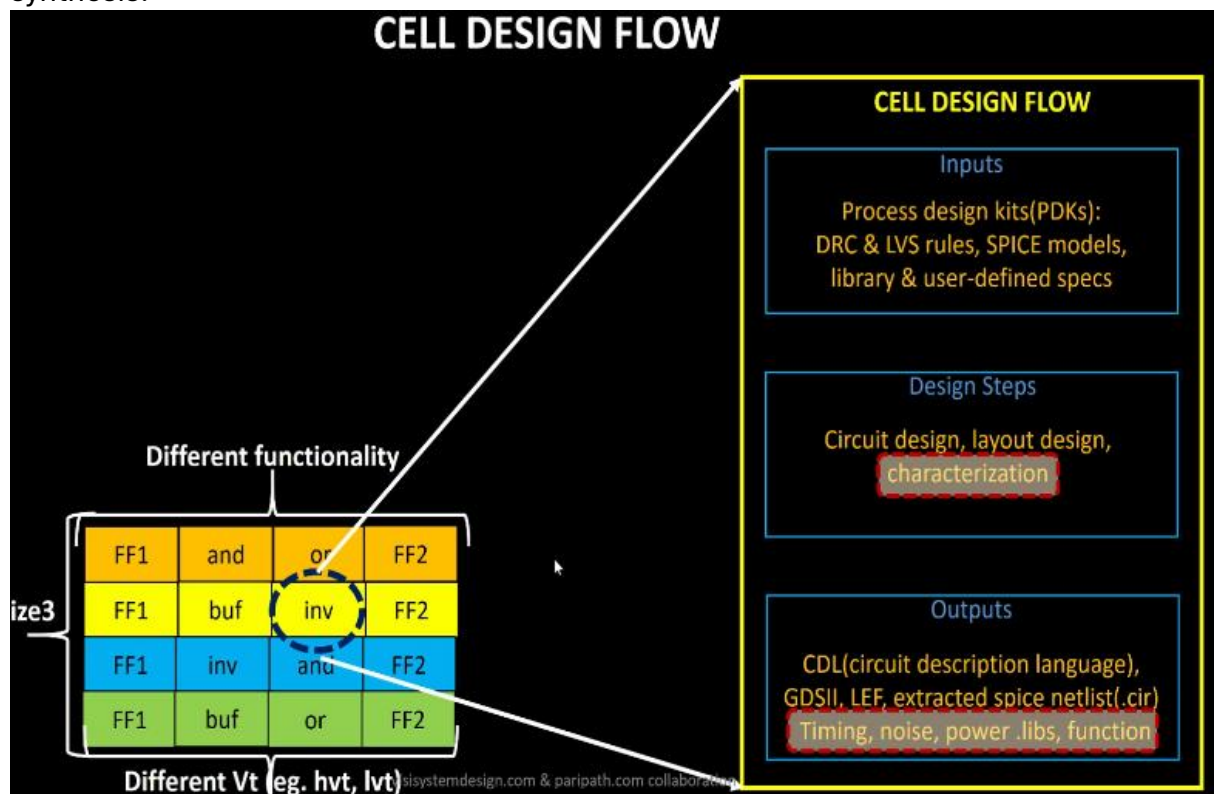
- **Layout Verification and Output Formats**

- The resulting hand-drawn layout transitions to a digital form using layout editing tools (e.g., open-source tool "magic"). Outputs typically include the GDSII (GDS2) layout file, which contains all geometrical information necessary for fabrication.

- GDSII format specifies component boundaries and facilitates manufacturing.
- The layout must comply with all DRC rules to ensure manufacturability.

- **Parasitic Extraction and Cell Characterization**

- Parasitic extraction derives the resistance and capacitance (R , C) of interconnects and devices directly from the drawn layout. This data is compiled into a SPICE netlist, integrating the extracted parasitic values.
- Extracted parameters are key for precise simulation of timing and electrical behavior.
- Characterization uses the SPICE netlist to compute:
 - Timing metrics
 - Noise margins
 - Power consumption
- The results are captured in standard output files (e.g., LIB, .lib files), defining the functionality and performance of the cell for broader use in digital circuit synthesis.



Key Points

- **Logical function** is implemented using PMOS/NMOS transistor networks specifically arranged to realize that function.
 - **PMOS and NMOS network graphs** clarify transistor connections and provide a basis for area-optimized transistor placement.
 - **Euler path** in the network graph yields a single continuous transistor ordering, minimizing diffusion breaks and yielding compact layouts.
 - **Stick diagrams** abstract transistor and interconnect arrangements before formal layout, bridging logical topology and physical design.
 - **Design Rule Constraints (DRC)** and user specifications rigorously define feature sizes, contact/metal placements, and cell boundaries.
 - **Layout editing tools** (e.g., magic) convert abstract representations into fabrication-ready GDSII files.
 - **Parasitic extraction** ensures realistic modeling of circuit performance by accounting for layout-induced electrical effects (R , C).
 - **Cell characterization** translates physical properties into timing, noise, and power metrics required for higher-level digital design automation.
-

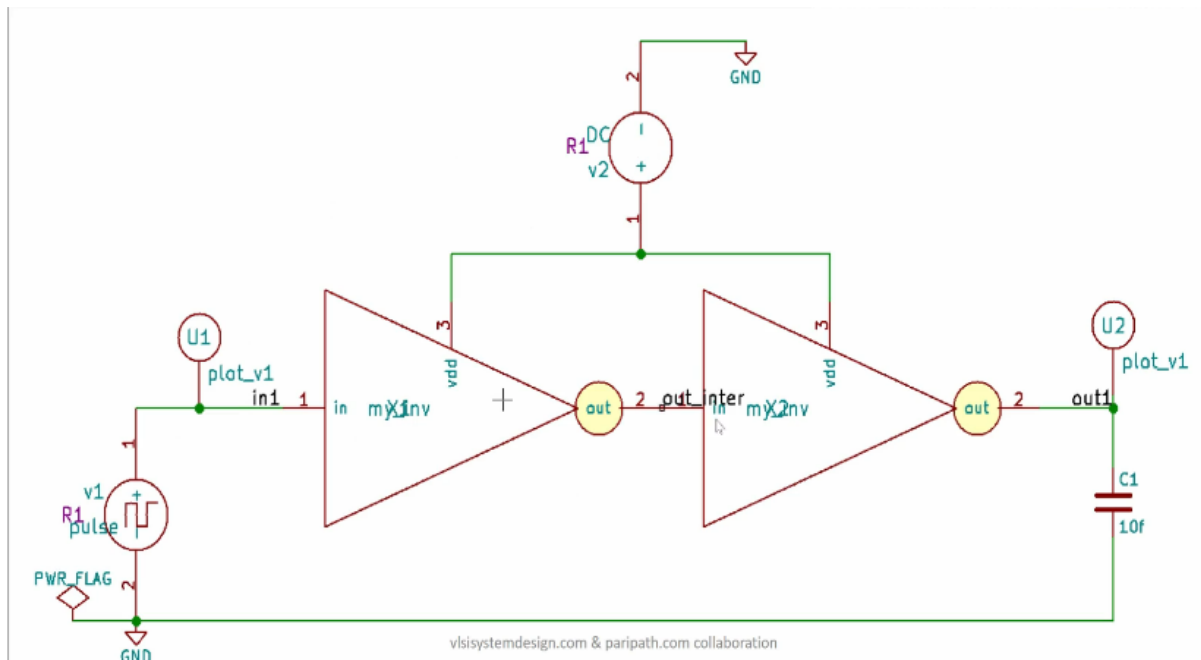
Typical characterization flow

This section covers the theoretical foundations and key ideas behind the cell-level characterization flow in custom digital design using open-source hardware tools. The focus is on understanding the role and nature of different inputs, the stepwise methodology followed in industry-standard flows, and the classification of resulting models used in timing, power, and noise analysis.

Core Concepts

Inputs to Characterization

- **Layout:** A fully designed geometric representation of a cell, such as a buffer or inverter, typically created with open-source physical layout tools. Each component in the layout (like metal layers and contacts) corresponds to electrical properties including resistance and capacitance.



- **Subcircuits and SPICE Models:** Subcircuits define reusable blocks such as inverters, instantiated multiple times in a design. SPICE device models (for NMOS and PMOS) represent the electrical characteristics of each transistor, capturing properties like threshold voltage (V_{TH}), doping, and process-specific behaviors.

```

.include my_inv.sub
v1 in1 gnd pulse=0 1.8 0 10p 10p 1n 2n)
* u1 in1 plot v1
* u2 out1 plot v1
v2 net- x1-pad3_gnd dc 1.8
x1 in1 net- x1-pad2_net- x1-pad3_my_inv
c1 out1 gnd 10f
x2 net- x1-pad2_out1 net- x1-pad3_my_inv
.tran 10e-12 4e-09 0e-00

* Control Statements
.control
run

```

Flow

```

.subckt my_inv in out vdd
.include NMOS-180nm.lib
.include PMOS-180nm.lib
m1 out in vdd vdd CMOSF W=0.9u L=0.18u M=1
m2 out in gnd gnd CMOSN W=0.36u L=0.18u M=1
* Control Statements
.ends my_inv

```

```

.model CMOSN NMOS (LEVEL=8 VERSION=3.2 TNOM=27 TOX=4.1E-9 XJ=1E-7 NCH=2.3540E17 VTH0=0.3823463 K1=0.5010697
K2=4.774618E-3 K3=0.0431669 K3B=1.1498346 W0=1E-7 NLX=1.910552E-7 DVT0W=0 DVT1W=0 DVT2W=0
DVT0=1.2894824 DVT1=0.3622063 DVT2=0.0713729 U0=280.633249 UA=-1.208537E-9 UB=2.158625E-18
UC=-5.342807E-11 VSAT=9.366802E4 A0=1.7593146 AGS=0.3939741 B0=-6.413949E-9 B1=-1E-7 KETA=-5.180424E-4
A1=0 A2=1 RDSW=105.5517558 PRWG=0.5 PRWB=-0.1998871 WR=1 WINT=7.904732E-10 LINT=1.571424E-8 XL=0
XM=-1E-8 DWG=1.297221E-9 DWB=1.479041E-9 VOFF=-0.0955434 NFACTOR=2.4358091 CIT=0 CDSC=2.4E-4 CD5CD=0
CDSCB=0 ETA0=3.104851E-3 ETAB=-2.512384E-5 DSUB=0.0167075 PCLM=0.8073191 PDIBLC1=0.1666161 PDIBLC2=3.112892E-3
PDIBLCB=0.1 DROUT=0.7875618 PSCBE1=8E10 PSCBE2=9.213635E-10 PVAG=3.85243E-3 DELTA=0.01 RSH=6.7 MOBMOD=1
PRT=0 UTE=-1.5 KT1=0.11 KT1L=0 KT2=0.022 UA1=4.31E-9 UB1=-7.61E-18 UC1=-5.6E-11 AT=3.3E4 WL=0 WLN=1
WN=0 WWN=1 WWL=0 LL=0 LLN=1 LW=0 LWN=1 LWL=0 CAPMOD=2 XPART=0.5 CGD0=7.08E-10 CGS0=7.08E-10 CGB0=1E-12
CJ=9.68858E-4 PB=0.8 MJ=0.3864502 CJSW=2.512138E-10 PBSW=0.809286 MJSW=0.1060414 CJSWG=3.3E-10 PBSWG=0.809286
MJSWG=0.1060414 CF=0 PVTH0=-1.192722E-3 PRDSW=-5 PK2=6.450505E-5 WKETA=-4.27294E-4 LKETA=-0.0104078
PU0=6.3268729 PUA=2.226552E-11 PUB=0 PVSAT=969.1480157 PETA0=1E-4 PKETA=-1.049509E-3)

```

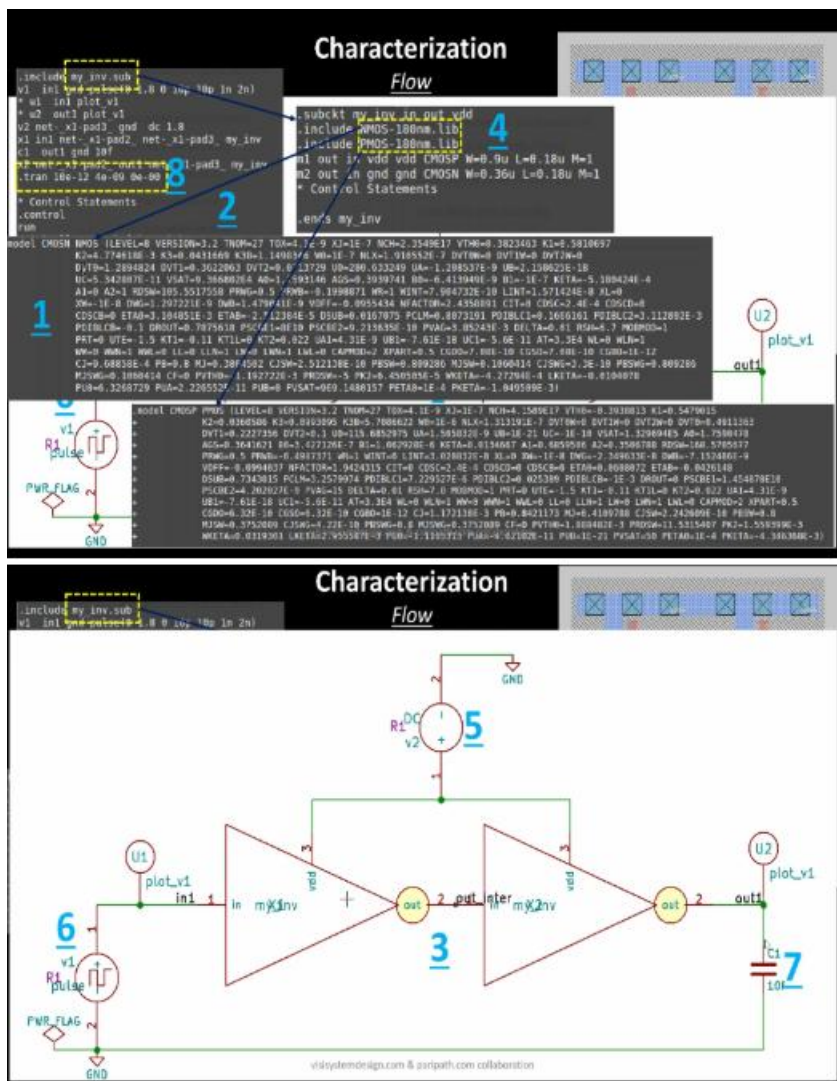
```

.include my_inv.sub
v1 in1 gnd pulse(0 1.8 0 10p 10p 1n 2n)
* u1 in1 plot v1
* u2 out1 plot v1
v2 net- x1-pad3_ gnd dc 1.8
x1 in1 net- x1-pad2_ net- x1-pad3_ my_inv
c1 out1 gnd 10f
x2 net- x1-pad2_ out1 net- x1-pad3_ my_inv
.tran 10e-12 4e-09 0e-00

* Control Statements
.control
run
print allv > plot_data v.txt
print alli > plot_data i.txt
plot v(in1)
plot v(out1)
.endc
.end

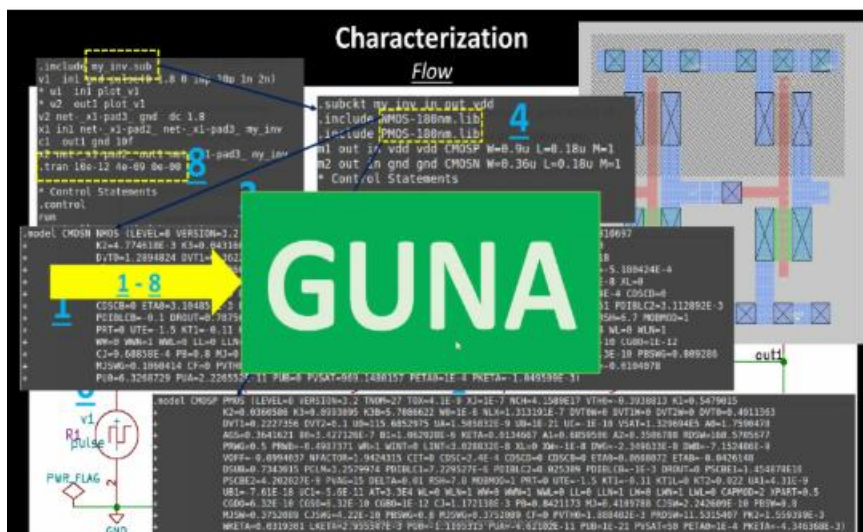
```

-
- **Characterization Flow: Stepwise Approach**
- A typical industry-standard cell characterization flow consists of these ordered steps:
 - **Read Model Files:** Import SPICE models for all device types (e.g., NMOS, PMOS), specifying their electrical characteristics.
 - **Read Extracted SPICE Netlist:** Load the complete netlist, including parasitic resistances and capacitances, detailing physical layout effects.
 - **Recognize Buffer Behavior:** Analyze the netlist to understand the logical and functional behavior of the cell (e.g., identify it as a buffer).
 - **Read Subcircuits:** Import subcircuit descriptions for any instantiated building blocks (such as inverters).
 - **Attach Power Sources:** Ensure all supply connections (e.g., V_{DD} , ground) are present and correctly tied to the cell.
 - **Apply Stimulus:** Define the types of input stimuli to drive the simulation, essential for exercising and characterizing circuit response.
 - **Specify Output Capacitance:** Configure external loading (e.g., a 10 fF output load) which may be varied for nonlinear delay or load-dependent measurements.
 - **Add Simulation Commands:** Choose the analysis type (e.g., transient, DC), specifying simulation directives (e.g., .tran) required for the characterization.
- These steps are typically compiled into a configuration file supplied to characterization software.



Role of Characterization Software

- The configuration file encapsulating all inputs is provided to a characterization tool (e.g., GUNA).
- The software uses these inputs to perform simulations and extract *timing*, *noise*, and *power* data.
- Generated models form the basis for further digital design flows and are essential for accurate design closure.



Model Classification

The resulting data from characterization flow is normally categorized as:

- **Timing Characterization:** Determines delays, setup/hold times, and signal propagation characteristics of the cell under varying conditions.
- **Power Characterization:** Quantifies the dynamic and static power consumption based on expected usage and switching patterns.
- **Noise Characterization:** Assesses the cell's behavior in the presence of noise, indicating susceptibility or robustness.

These comprehensive models are critical for assembling higher-level digital blocks and ensuring reliable design at system scale.

Key Points

- Accurate cell characterization relies on integrating layout, netlist extraction (including physical parasitics), libraries of subcircuits, and accurate SPICE models for devices.
- The characterization flow is a formalized, stepwise process culminating in detailed functional, power, and noise models applicable in subsequent simulation and analysis.
- Output models (often in the form of library files such as .lib) are divided by the type of analysis: timing, power, and noise.
- All simulations depend fundamentally on the integrity of the device models and quality of the physically extracted netlist.

- Stimulus and load selection are critical: choices made at this stage impact the representativeness and utility of generated timing and power models.

• General timing characterization parameters

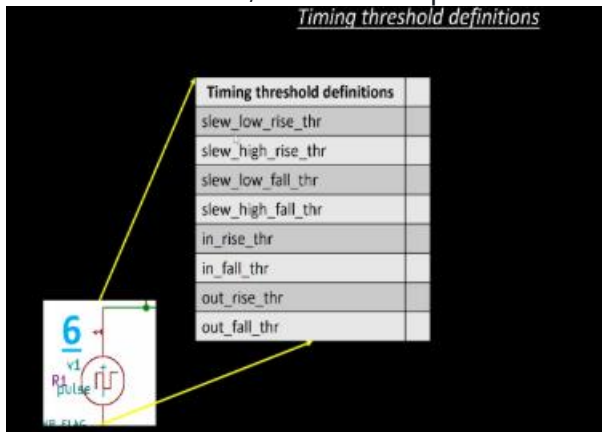
• Timing threshold definitions

- This document introduces the theoretical concepts and syntax used in timing characterization, specifically for digital circuits utilizing inverters and buffers. Core definitions include timing thresholds, slew rate metrics, and propagation delay, all essential parameters for timing analysis and simulation in characterization software.

• Core Concepts

• Timing Characterization Syntax and Variables

- Timing characterization relies on a set of defined variables and thresholds that describe the behavior of digital waveforms at different stages in a circuit (e.g., inverter outputs, buffer outputs). These variables are critical inputs for characterization software, influencing further computations such as delays, current estimates, and overall performance metrics.

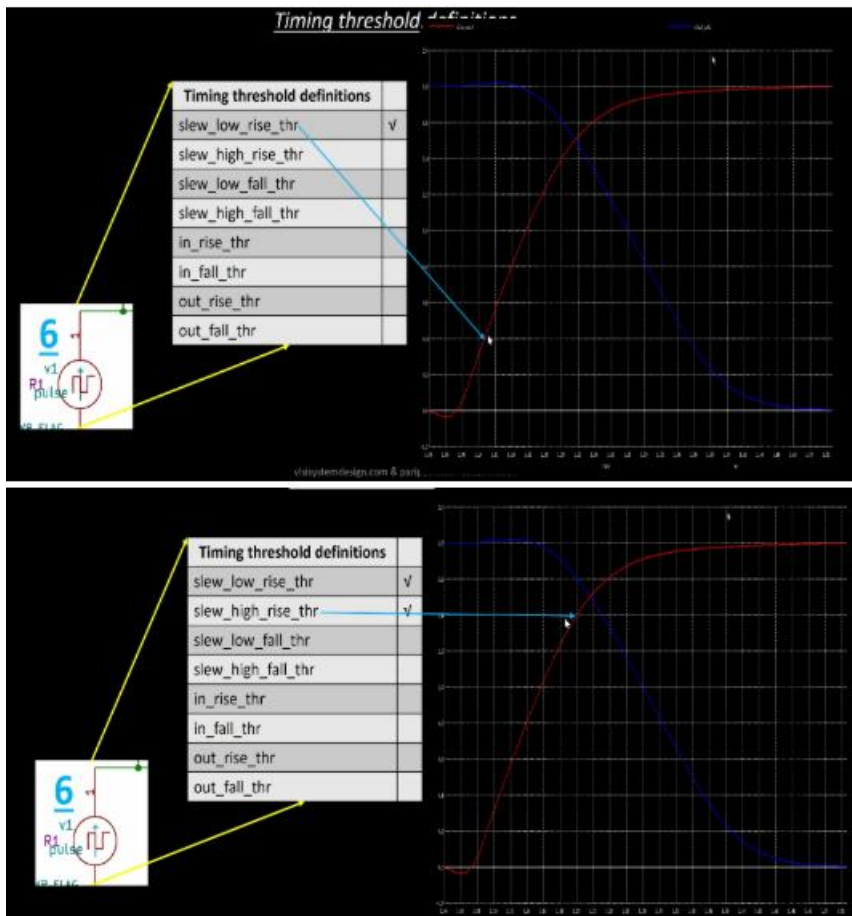


• Timing Threshold Definitions

• Threshold Points on a Waveform

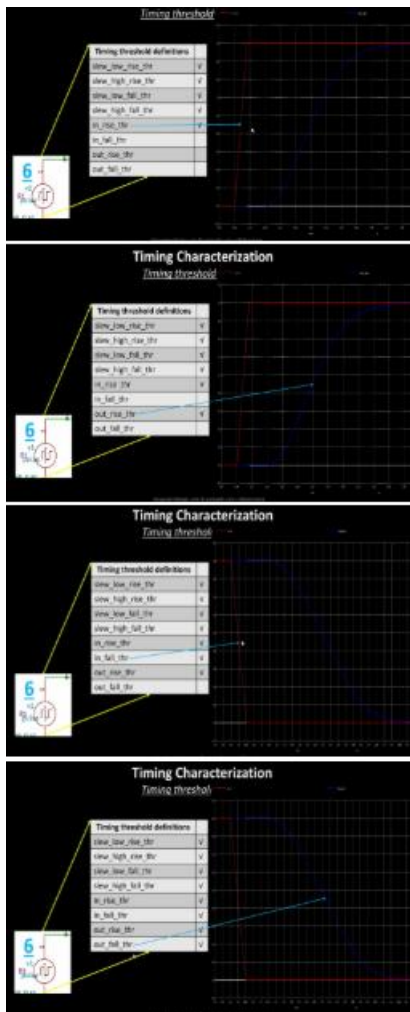
- Digital waveforms undergo transitions between logic states, and specific percentage levels of the supply voltage are used as **thresholds** to mark these state changes. Core thresholds include:
 - **Slew Low Threshold:** A value near the lower supply limit, typically $\sim 0.2V_{DD}$ or 20% of the supply voltage, defining the start of a rising transition.
 - **Slew High Threshold:** A value near the upper supply limit, often $\sim 0.8V_{DD}$ or 80% of the supply voltage, defining the end of a rising transition.

- The same thresholds apply to falling transitions but refer to falling from high to low, with **slew low fall** and **slew high fall** thresholds.
- These thresholds are consistently applied to both input and output waveforms and form the basis of slew and delay calculations.



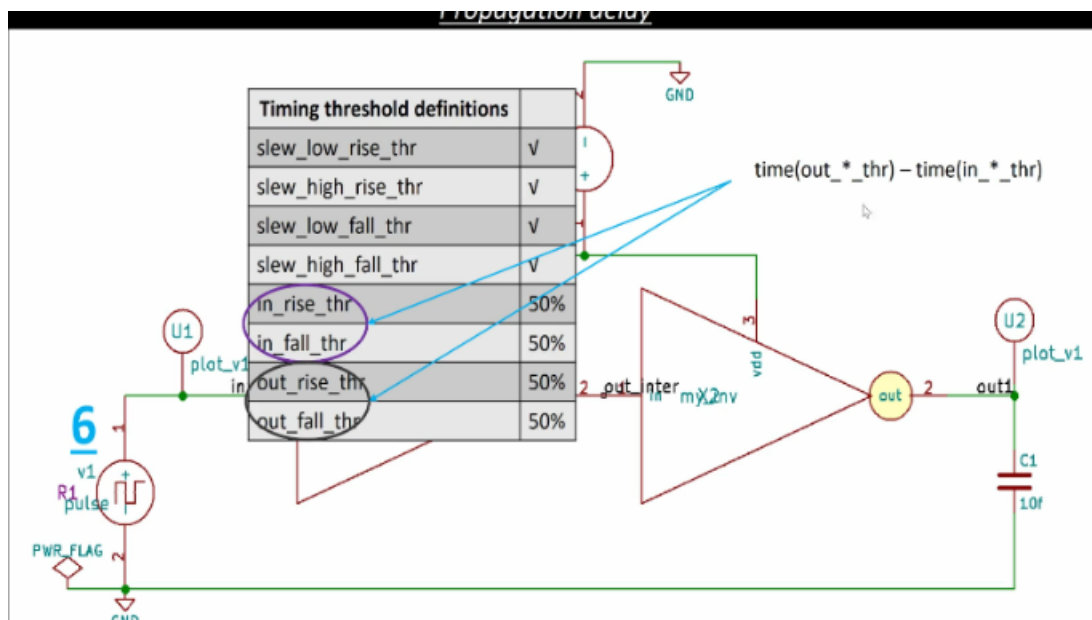
• Slew Calculation

- Slew, or transition rate, is calculated using the time difference between the two threshold crossings:
- For a rising edge: $\text{slew}_{\{\text{rise}\}} = t_{\{\text{high threshold}\}} - t_{\{\text{low threshold}\}}$
- Example: If 20% and 80% are used,
- Extra open brace or missing close brace
-
- $\text{slew}_{\{\text{rise}\}} = t_{\{80\}} - t_{\{20\}}$
- This metric quantifies how fast the signal transitions and is key for noise analysis, power estimation, and reliability predictions.



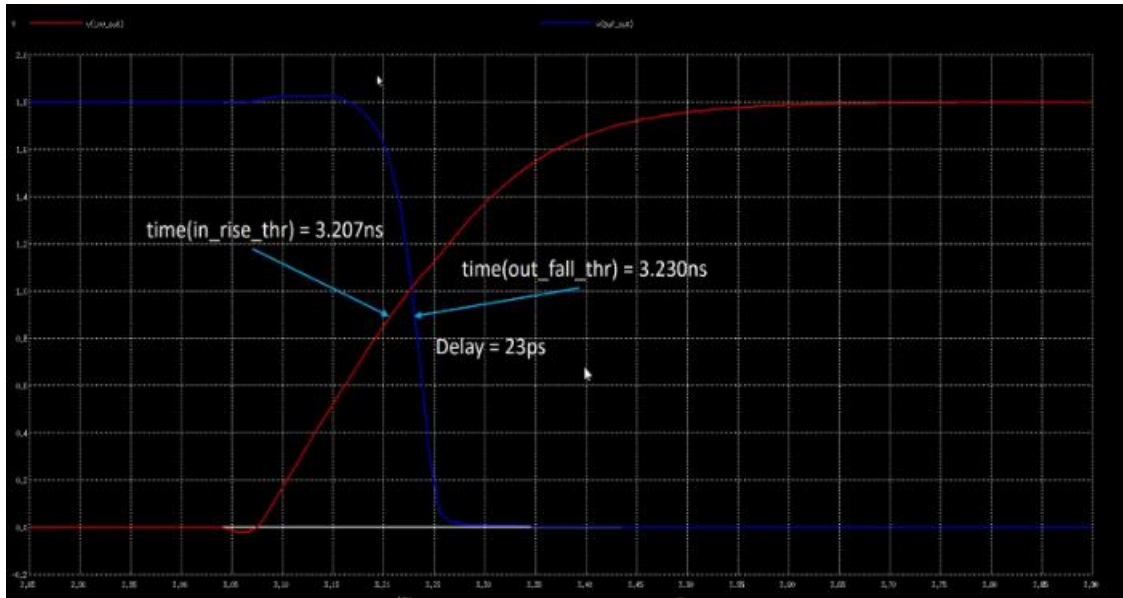
-
- **propagation Delay Concept**
- Propagation delay quantifies how long a signal takes to travel through a logic element (such as an inverter or buffer). It is calculated as:
 - For rising transitions: $t_{\text{delay,~rise}} = t_{\text{out, rise(50\%)}} - t_{\text{in, rise(50\%)}}$
 - For falling transitions: $t_{\text{delay,~fall}} = t_{\text{out, fall(50\%)}} - t_{\text{in, fall(50\%)}}$
 - This delay is crucial for evaluating circuit timing, setup and hold margins, and overall system performance.
- **Key Points**
- **Thresholds** define key points on waveforms (e.g., 20%, 50%, 80% of V_{DD}) used to consistently characterize behavior during signal transitions.
- **Slew rate** measures the speed of signal transitions and requires both low and high threshold points.
- **Input/Output thresholds** for rising or falling waveforms are usually set at the 50% mark and serve as reference points for measuring delays.
- **Propagation delay** is the time difference between input and output threshold crossings and forms a fundamental parameter in timing analysis.

- The definitions and metrics outlined here serve as **fundamental building blocks** for further timing characterization, simulation, and performance prediction in digital design.
- **Propagation delay and transition time**
- This documentation covers the theoretical aspects of **propagation delay** and **transition (slew) time** in digital circuit timing characterization. It addresses their definitions, calculation methods, the importance of threshold choices, and key design considerations such as wire delays and their effects on accurate delay measurement.
- **Core Concepts**
- **Propagation Delay**
- **Propagation delay** is the time interval between a change at the input of a circuit and the corresponding change observed at its output. This value is essential in determining circuit performance and in ensuring reliable timing across digital systems.
- The general formula for propagation delay is: $delay = t_{output,threshold} - t_{input,threshold}$
- Thresholds are specific voltage points—commonly expressed as a percentage of V_{DD} (e.g., 50%)—used to determine when the input or output waveform is considered to have transitioned.



Importance of Threshold Choice

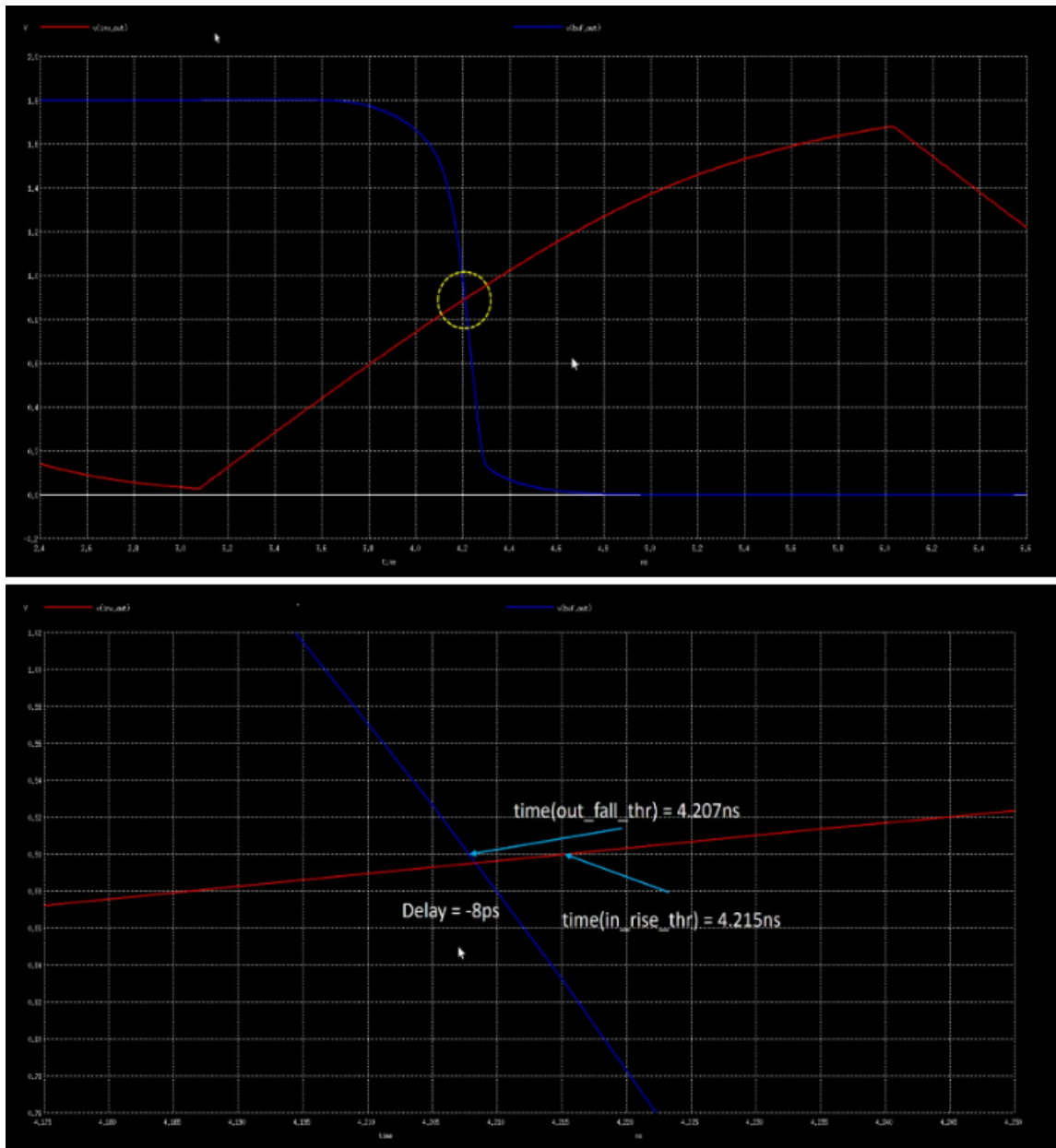
- Selecting **inconsistent or non-standard thresholds** can result in observing a *negative* propagation delay, which does not physically make sense and usually indicates poor measurement practice.
- Standard practice is to use the 50% point of V_{DD} for both input and output as the threshold. However, both points must be selected carefully and consistently.



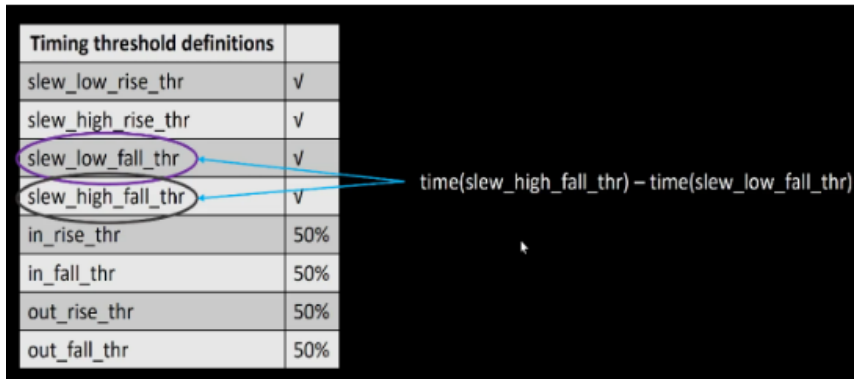
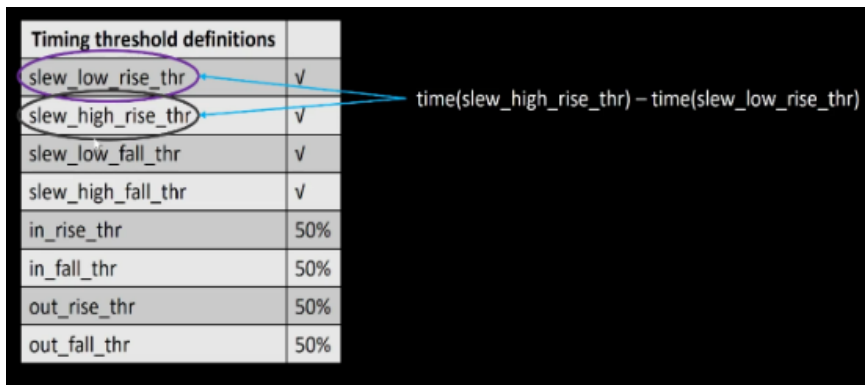
Effects of Physical Circuit Characteristics

- Large distances or poor layout between gates (e.g., long wires) introduce additional delay components. This can distort waveform shapes and even (with correct thresholds) sometimes produce apparent negative delays due to heavily skewed waveforms.

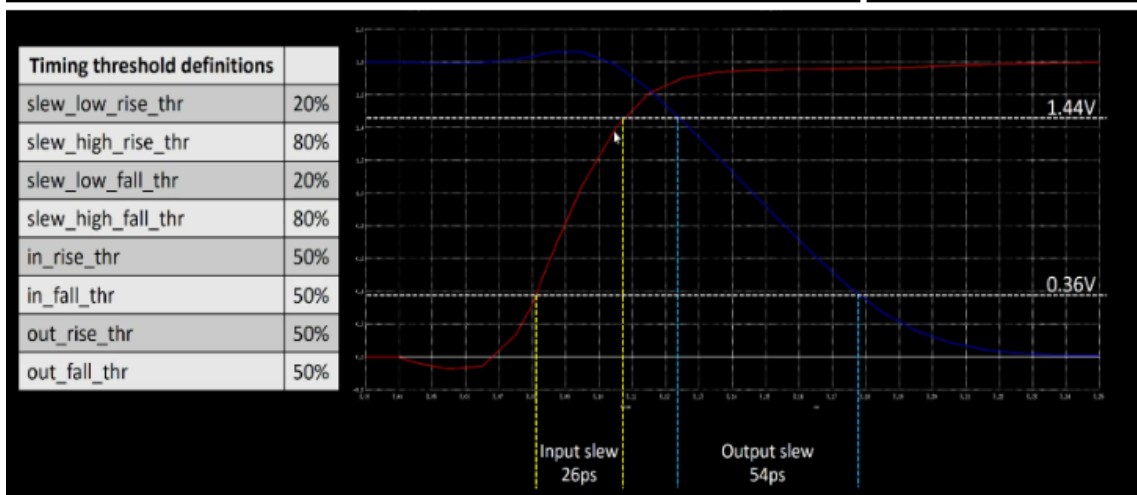
-



- **Transition (Slew) Time**
- **Transition time (or slew)** measures how quickly a signal rises or falls between two voltage thresholds.
- For rising transitions: $slew_{rise} = t_{high,threshold} - t_{low,threshold}$
- For falling transitions: $slew_{fall} = t_{high,threshold} - t_{low,threshold}$
- Typical industrial thresholds for slew measurement are 20% and 80% of V_{DD} . For example, if $V_{DD} = 1.8, V$, then 20% is $0.36, V$ and 80% is $1.44, V$.



Timing threshold definitions	
slew_low_rise_thr	20%
slew_high_rise_thr	80%
slew_low_fall_thr	20%
slew_high_fall_thr	80%
in_rise_thr	50%
in_fall_thr	50%
out_rise_thr	50%
out_fall_thr	50%



- The formula is always *high* – *low* for both rising and falling cases, ensuring a positive value for transition time.
- **Key Points**
- **Propagation delay** is calculated as the difference between when the output and input waveforms cross specified thresholds: $\text{delay} = t_{\text{out},th} - t_{\text{in},th}$.
- **Standardization** in threshold selection (typically 50% of V_{DD}) is essential to avoid incorrect or negative delay values.
- **Negative delay values** may arise from either poor threshold selection or significant wire/waveform distortion, both of which indicate issues in measurement or design.
- **Transition/slew time** quantifies how quickly a signal changes, using defined low and high thresholds (commonly 20% and 80% of V_{DD}).

- Correct circuit characterization requires attention to both accurate threshold selection and the underlying physical design, as both can dramatically impact observed timing metrics.